

HTW Berlin

Hochschule für Technik und Wirtschaft Berlin

Studiengang Computer Engineering

Bachelorarbeit

**Untersuchung von Watchdog- und
Recovery-Strategien
für ein robustes eingebettetes System**

Verfasser:	Omar Altanbakji
Matrikelnummer:	S0587682
Studiengang:	Computer Engineering
Erstgutachter:	Prof. Dr. Jochen Kerdels
Zweitgutachter:	Prof. Dr. Marcin Brzozowski
Abgabedatum:	21. September 2026

Zusammenfassung

Diese Bachelorarbeit untersucht Watchdog- und Recovery-Strategien an einem heterogenen eingebetteten System aus Raspberry Pi und STM32L432KC. Als Testanwendung erfasst der Raspberry Pi Positionsdaten eines Binghe VK-162 USB-GPS-Empfängers, ruft Wetterdaten für den ermittelten Standort ab und überträgt einen strukturierten Status über UART. Der STM32 prüft die Datenintegrität, stellt Nutz- und Fehlerzustände auf einem LCD dar und überwacht seine Komponenten mit einem lokalen Supervisor.

Die Architektur kombiniert mehrere Schutzebenen: Protokollvalidierung, lokale Komponenten-Recovery, degradierte Betriebsmodi, getrennte systemd-Dienste mit Restart-Policy und Service-Watchdog, den Independent Watchdog des STM32 sowie den Hardware-Watchdog des Raspberry Pi. Die Zuordnung folgt dem Prinzip, die kleinste Recovery-Ebene zu verwenden, die den jeweiligen Fehler zuverlässig beseitigt.

Die Evaluation umfasst reale Netzwerkunterbrechungen, den Verlust des GPS-Empfängers, die Unterbrechung der UART-Verbindung, Dienst-Hänger und Dienst-Abstürze, einen absichtlichen STM32-Firmware-Hänger, fehlerhafte und übergroße UART-Nachrichten, einen LCD-/I2C-Ausfall, vollständige Spannungsverluste, einen kontrollierten Unterspannungstest auf Platineebene und einen simulierten Kernel-Hänger. Die wiederholten funktionalen Recovery-Testfälle wurden jeweils fünfmal ausgeführt und waren innerhalb des untersuchten Umfangs in allen Durchläufen erfolgreich. Die mittlere End-to-End-Fehlerweitergabe bei einer blockierten HTTPS-Verbindung betrug 7,36 s, die anschließende funktionale Recovery 10,10 s. Ein STM32-Firmware-Hänger wurde durch den IWDG in 5,70 s bis zur Rückkehr in den Anwendungsmodus behoben. Ein hängender GPS-Dienst benötigte im Mittel 28,46 s, ein abgestürzter Dienst dagegen 5,76 s. Nach vollständigem Spannungsverlust betrug die extern gemessene Netzwerk-Nichtverfügbarkeit im Mittel 32,33 s.

Der ergänzende Unterspannungstest am VIN-Eingang der NUCLEO-L432KC-Platine zeigte in fünf von fünf Durchläufen denselben stufenweisen Funktionsverlust: Normalbetrieb bei 8–6 V, verringerte LCD-Hintergrundbeleuchtung bei 5 V, Ausfall der Zeichendarstellung mit STM32-Neustart bei 4 V, vollständigen LCD-Ausfall mit erneutem Neustart bei 3 V und vollständigen Stillstand des STM32 bei 2 V. Beim Raspberry Pi belegte der Statuswert 0x50005 bei einer am Labornetzteil eingestellten Spannung von 4,50 V gleichzeitig aktive Unterspannung und Drosselung. Die Ergebnisse zeigen, dass eine abgestufte Fehlerbehandlung unnötige Gesamtneustarts vermeidet und in den durchgeführten Versuchen auch schwere Hänger behandeln konnte. Als Grenzen verbleiben insbesondere die manuelle Fehlerinjektion, das Hot-Start-Verhalten des GPS-Empfängers sowie die fehlende Messung der internen Versorgungsschienen und Reset-Ursachen während des Unterspannungstests.

Abkürzungsverzeichnis

API	Application Programming Interface
CRC	Cyclic Redundancy Check
GPIO	General Purpose Input/Output
HTTP(S)	Hypertext Transfer Protocol (Secure)
BOR	Brown-out Reset
I2C	Inter-Integrated Circuit
ICMP	Internet Control Message Protocol
IWDG	Independent Watchdog
JSON	JavaScript Object Notation
LCD	Liquid Crystal Display
LSI	Low-Speed Internal Oscillator
NMEA	National Marine Electronics Association
PVD	Programmable Voltage Detector
RTC	Real-Time Clock
UART	Universal Asynchronous Receiver/Transmitter
WDT	Watchdog Timer

Inhaltsverzeichnis

Zusammenfassung	I
Abkürzungsverzeichnis	II
1. Einleitung	1
1.1. Motivation	1
1.2. Problemstellung	1
1.3. Zielsetzung	1
1.4. Forschungsfragen	2
1.5. Aufbau der Arbeit	2
2. Theoretische Grundlagen	3
2.1. Eingebettete Systeme und heterogene Fehlerdomänen	3
2.2. Dependability und Robustheit	3
2.3. Fault, Error und Failure	3
2.4. Fehlerbehandlung als Folge von Phasen	4
2.5. Recovery-orientierter Entwurf	4
2.6. Grundprinzip eines Watchdogs	4
2.7. Watchdog-Ebenen	5
2.7.1. Anwendungs- und Kommunikationsüberwachung	5
2.7.2. Service-Watchdog	5
2.7.3. Mikrocontroller-Watchdog	5
2.7.4. Systemweiter Hardware-Watchdog	5
2.8. Timeout-Dimensionierung	5
2.9. Heartbeat und Supervisor	6
2.10. Datenintegrität serieller Kommunikation	6
2.11. I2C und lokale Bus-Recovery	6
2.12. GPS, NMEA und externe Wetterdaten	6
2.13. Messung von Recovery-Zeiten	7
2.14. Zwischenfazit	7
3. Anforderungen und Systemkonzept	8
3.1. Anwendungsfall als Testumgebung	8
3.2. Funktionale Anforderungen	8
3.3. Nichtfunktionale Anforderungen	9
3.4. Fehlermodell	9
3.5. Systemarchitektur	10
3.5.1. Aufgabenteilung	10
3.5.2. Dienstarchitektur auf dem Raspberry Pi	10
3.5.3. Architektur auf dem STM32	10
3.6. Konzept der UART-Verbindung	11
3.6.1. Heartbeat	11
3.6.2. Datenrahmen	11
3.6.3. Betriebsmodi	11
3.7. Hierarchisches Watchdog- und Recovery-Konzept	11
3.8. Systemweiter Recovery-Zustandsautomat	12
3.9. Erfolgskriterien des Gesamtkonzepts	14

4. Implementierung	15
4.1. Hardwareaufbau	15
4.2. Raspberry-Pi-Softwarearchitektur	15
4.2.1. Gemeinsame Konfiguration	15
4.2.2. Zustandsaustausch und atomare Aktualisierung	16
4.2.3. GPS-Dienst	16
4.2.4. Netzwerkdienst	16
4.2.5. UART-Dienst und Protokollbildung	16
4.3. systemd-Integration	17
4.4. Hardware-Watchdog des Raspberry Pi	17
4.5. STM32-Firmwarestruktur	17
4.5.1. Bootablauf und Diagnose	18
4.5.2. Supervisor und Gesundheitsmodell	18
4.5.3. Independent Watchdog	18
4.5.4. UART-Empfang und Fehlergrenzen	19
4.5.5. LCD- und I2C-Recovery	19
4.5.6. Spannungsüberwachung	19
4.6. Diagnose, Logs und Reproduzierbarkeit	19
5. Evaluation	20
5.1. Zielsetzung und Evaluationsfragen	20
5.2. Versuchsaufbau	20
5.2.1. Hardwareaufbau	20
5.2.2. Softwarearchitektur des Raspberry Pi	21
5.2.3. Softwarearchitektur des STM32	22
5.2.4. Hierarchie der Überwachungs- und Recovery-Ebenen	22
5.3. Messmethodik	22
5.3.1. Messgrößen	22
5.3.2. Monotone Zeitquelle und Journal-Cursor	23
5.3.3. Externe Messung bei vollständigen Neustarts	23
5.3.4. Manuell ausgelöste Hardwarefehler	23
5.3.5. Wiederholungen und statistische Kenngrößen	23
5.3.6. Dokumentation und Datenintegrität	24
5.4. Definition der Testfälle	24
5.5. Messergebnisse	25
5.5.1. Gesamtübersicht	25
5.5.2. N2: Ausfall und Wiederherstellung der Wetterverbindung	27
5.5.3. G1: Verlust und Wiederkehr des GPS-Empfängers	27
5.5.4. C1: Unterbrechung der Verbindung zum STM32	27
5.5.5. H1: Firmware-Hänger und STM32-IWDG	27
5.5.6. S1 und S2: Hängender gegenüber abgestürztem GPS-Dienst	28
5.5.7. S3: Hänger des UART-Dienstes	28
5.5.8. U1 und U2: UART-Datenintegrität und Puffergrenzen	28
5.5.9. L1: Lokale Recovery des LCD-/I2C-Pfads	29
5.5.10. P1: Vollständiger Spannungsverlust	29
5.5.11. P2: Kernel-Hänger und Hardware-Watchdog	30
5.5.12. P3: Kontrollierter Unterspannungstest	30
5.6. Zusammenfassung der Messergebnisse	31
6. Diskussion	33

6.1. Beantwortung der Forschungsfragen	33
6.1.1. F1: Strukturierung und geeignete Fehlerdomäne	33
6.1.2. F2: Eingriffstiefe und Wiederherstellungszeit	33
6.1.3. F3: Absturz, Hänger, Firmware-Hänger und System-Hänger	34
6.1.4. F4: Automatische Rückkehr zum definierten Zustand	34
6.2. Dienst-Hänger gegenüber Dienst-Absturz	34
6.3. Nutzen des degradierten Betriebs	34
6.4. Lokale Recovery und Zustandsverlust	35
6.5. Datenintegrität und Grenzen der XOR-Prüfsumme	35
6.6. Trade-off der Watchdog-Zeiten	35
6.7. Spannungsverhalten	35
6.8. Validität und Messunsicherheit	36
6.8.1. Interne Validität	36
6.8.2. Konstruktvalidität	36
6.8.3. Externe Validität	36
6.9. Gesamtbewertung	36
7. Fazit und Ausblick	37
7.1. Fazit	37
7.2. Ausblick	37
Literaturverzeichnis	39

Abbildungsverzeichnis

Abbildung 3.1: Gesamtarchitektur und wesentliche Datenpfade des Versuchssystems	8
Abbildung 3.2: Entkopplung der Raspberry-Pi-Dienste über atomar aktualisierte Zustandsdateien	10
Abbildung 3.3: Eskalationshierarchie der Recovery-Strategien	12
Abbildung 3.4: Systemweiter Zustandsautomat für Fehlerbehandlung und Recovery	13
Abbildung 4.1: Vereinfachter Boot- und Recovery-Ablauf des STM32	18
Abbildung 5.1: Vergleich der mittleren funktionalen Wiederherstellungszeiten	28
Abbildung 5.2: Extern gemessene Ausfallzeiten der drei P1-Durchläufe	29
Abbildung 5.3: Qualitative Degradationsstufen in Abhängigkeit von der am Labornetzteil eingestellten VIN-Spannung	30
Abbildung 5.4: Raspberry-Pi-Status bei 4,50 V: throttled=0x50005	31

Tabellenverzeichnis

Tabelle 3.1: Funktionale Anforderungen	8
Tabelle 3.2: Nichtfunktionale Anforderungen	9
Tabelle 3.3: Fehlermodell und vorgesehene Recovery-Strategien	9
Tabelle 3.4: Aufgabenteilung zwischen Raspberry Pi und STM32	10
Tabelle 3.5: Übergänge des systemweiten Recovery-Zustandsautomaten	14
Tabelle 4.1: Wesentliche Schnittstellen und Konfigurationswerte	15
Tabelle 5.1: Hardwarekomponenten des Versuchsaufbaus	21
Tabelle 5.2: Übersicht der untersuchten Testfälle (Teil 1)	24
Tabelle 5.2: Übersicht der untersuchten Testfälle (Teil 2)	25
Tabelle 5.3: Zusammenfassung der zentralen Messergebnisse	26
Tabelle 5.4: Externe Netzwerknichtverfügbarkeit bei vollständigem Spannungsverlust	29
Tabelle 5.5: Reproduzierbares Unterspannungsverhalten der NUCLEO-L432KC- Platine	31
Tabelle 6.1: Zuordnung der untersuchten Fehlerklassen zu Recovery-Ebenen	33

1. Einleitung

1.1. Motivation

Eingebettete Systeme übernehmen klar abgegrenzte Funktionen innerhalb technischer Anlagen und arbeiten häufig lange ohne unmittelbare menschliche Überwachung. Ihre Verfügbarkeit hängt nicht allein von der eigentlichen Anwendungslogik ab. Sensoren und Aktoren, Kommunikationspfade, Betriebssystemdienste, externe Server und die Energieversorgung bilden zusätzliche Fehlerdomänen. Eine Implementierung, die nur den Normalfall berücksichtigt, kann bereits durch einen vorübergehenden Netzwerkausfall oder einen blockierten Prozess dauerhaft unbrauchbar werden. Robustheit bedeutet in diesem Zusammenhang nicht, Fehler vollständig zu verhindern. Entscheidend ist, dass Fehler erkannt, eingegrenzt und mit einer zur Fehlerklasse passenden Maßnahme behandelt werden. Die Dependability-Taxonomie unterscheidet unter anderem Verfügbarkeit, Zuverlässigkeit, Integrität und Wartbarkeit als Eigenschaften, die Vertrauen in ein System begründen [1, S. 13–14]. Recovery-orientierte Ansätze verschieben den Entwurfsfokus zusätzlich von der ausschließlichen Vermeidung von Fehlern hin zu einer schnellen und überprüfbaren Wiederherstellung [2, S. 1]. Ein pauschaler Neustart des Gesamtsystems ist zwar einfach, aber nicht für jede Störung angemessen. Ein ungültiger UART-Datenrahmen kann lokal verworfen werden. Ein nicht kritischer Displayausfall kann zu einem degradierten Betrieb führen, ohne die übrige Funktion zu unterbrechen. Ein Firmware-Hänger des Mikrocontrollers erfordert dagegen einen Reset, während ein Kernel-Hänger des Linux-Systems nur durch eine vom betroffenen Softwarepfad unabhängige Instanz behoben werden kann. Daraus entsteht die Notwendigkeit einer abgestuften Watchdog- und Recovery-Architektur.

1.2. Problemstellung

Die zentrale Problemstellung besteht darin, geeignete Überwachungs- und Wiederherstellungsmechanismen für unterschiedliche Fehlerklassen in einem heterogenen eingebetteten System auszuwählen und ihre Wirksamkeit experimentell zu bewerten. Das betrachtete System umfasst einen Raspberry Pi mit Linux, einen STM32-Mikrocontroller, einen Binghe VK-162 USB-GPS-Empfänger, eine Internetverbindung und ein LCD. Fehler können daher auf Daten-, Komponenten-, Prozess-, Firmware- und Systemebene auftreten. Die technische Herausforderung liegt nicht nur im Aktivieren einzelner Watchdogs. Ein Prozessabsturz ist für den Service-Manager unmittelbar sichtbar, während ein weiterhin vorhandener, aber hängender Prozess erst durch ausbleibende Fortschrittsnachweise erkannt wird. Ein Watchdog-Lebenszeichen darf zudem nicht bedingungslos erzeugt werden, weil sonst ein funktionierender Timer einen Stillstand der eigentlichen Nutzfunktion verdecken könnte. Schließlich soll eine Recovery möglichst nur die tatsächlich betroffene Fehlerdomäne zurücksetzen.

1.3. Zielsetzung

Ziel dieser Arbeit ist die Untersuchung, Implementierung und experimentelle Bewertung mehrerer Watchdog- und Recovery-Strategien für ein heterogenes System aus Raspberry Pi und STM32. Als Testanwendung bestimmt der Raspberry Pi eine Position über GPS, ruft Wetterdaten ab und überträgt den Zustand an den STM32, der die Daten

auf einem LCD darstellt. Die Wetteranwendung ist Mittel zum Zweck: Sie erzeugt einen nachvollziehbaren Datenpfad mit realen externen Abhängigkeiten und erlaubt die gezielte Fehlerinjektion. Untersucht werden die Auswahl der Recovery-Ebene, die Erkennungs- und Wiederherstellungszeiten sowie die Frage, ob der definierte funktionale Endzustand ohne manuellen Neustart erneut erreicht wird.

1.4. Forschungsfragen

F1: Wie muss ein heterogenes eingebettetes System strukturiert werden, damit Fehler auf der jeweils geeigneten Ebene erkannt und behandelt werden?

F2: Wie unterscheiden sich lokale Recovery, Dienstneustart, Mikrocontroller-Reset und vollständiger Systemneustart hinsichtlich Eingriffstiefe und funktionaler Wiederherstellungszeit?

F3: Welche Unterschiede zeigen sich zwischen Prozessabsturz, Prozess-Hänger, Firmware-Hänger und vollständigem Linux-Hänger?

F4: In welchem Umfang kann das entwickelte System nach den betrachteten Hardware-, Kommunikations- und Softwarefehlern automatisch in einen definierten Zustand zurückkehren?

1.5. Aufbau der Arbeit

Kapitel 2 führt Dependability, Watchdogs, Supervisoren und Recovery-Strategien ein. Kapitel 3 leitet Anforderungen, Fehlermodell und Recovery-Hierarchie ab. Die konkrete Umsetzung auf Raspberry Pi und STM32 wird in Kapitel 4 beschrieben. Kapitel 5 dokumentiert Fehlerinjektion, Messmethodik und Ergebnisse. Kapitel 6 ordnet die Ergebnisse ein und beantwortet die Forschungsfragen. Kapitel 7 fasst die Erkenntnisse zusammen und nennt Erweiterungsmöglichkeiten.

2. Theoretische Grundlagen

2.1. Eingebettete Systeme und heterogene Fehlerdomänen

Ein eingebettetes System ist in eine technische Umgebung integriert und stellt eine spezialisierte Funktion bereit. In einem heterogenen Aufbau können ein Linux-Rechner, ein Mikrocontroller, Kommunikationsschnittstellen und externe Dienste gleichzeitig beteiligt sein. Diese Teile besitzen unterschiedliche Laufzeitmodelle und Fehlermechanismen. Ein Linux-Dienst kann abstürzen oder blockieren, ein Mikrocontroller kann in einer Endlosschleife verbleiben, eine serielle Verbindung kann Daten verlieren und ein externer Dienst kann zeitweise nicht erreichbar sein. Für die Fehlerbehandlung ist deshalb die Definition von Fehlerdomänen hilfreich. Eine Fehlerdomäne umfasst Komponenten, die durch dieselbe Fehlerursache gemeinsam ausfallen können. Ein Software-Watchdog im Linux-Prozess ist gegenüber Kernel-Hängern nicht unabhängig [3, Abschn. „The simplest API“]. Der IWDG des STM32 arbeitet unabhängig von Hauptschleife und Haupttakt, da ihn der separate LSI-Oszillator taktet [4, S. 1037-1045, Abschn. 33].

2.2. Dependability und Robustheit

Avizienis et al. fassen unter Dependability Eigenschaften zusammen, die ein begründetes Vertrauen in einen Dienst ermöglichen. Dazu gehören Verfügbarkeit, Zuverlässigkeit, Safety, Integrität und Wartbarkeit [1, S. 13-14]. Für die vorliegende Arbeit wird Robustheit operational durch folgende Eigenschaften beschrieben:

1. Ein Fehler wird in einem nachvollziehbaren Zeitfenster erkannt.
2. Der Fehler wird in einen definierten internen und extern sichtbaren Zustand übersetzt.
3. Nicht betroffene Komponenten bleiben nach Möglichkeit aktiv.
4. Eine zur Fehlerklasse passende Recovery-Maßnahme wird ausgeführt.
5. Die Rückkehr zu einer funktionsfähigen Konfiguration ist durch Logs, Statusdateien oder Diagnosemeldungen nachweisbar. Robustheit ist damit nicht gleichbedeutend mit vollständiger Fehlertransparenz. Ein sichtbarer Zustand wie NET=ERR kann robuster sein als die unmarkierte Darstellung veralteter Wetterwerte, weil die Datenqualität korrekt kommuniziert wird.

2.3. Fault, Error und Failure

Die Dependability-Taxonomie unterscheidet eine Kette aus Fault, Error und Failure [1, S. 13 und 20–22]. Ein Fault ist die Ursache, etwa eine getrennte SDA-Leitung oder ein blockierter Prozess. Der Fault kann einen fehlerhaften internen Zustand erzeugen, beispielsweise ein I2C-NACK oder einen veralteten JSON-Zustand. Wird die geforderte Funktion an der Systemgrenze nicht mehr erbracht, liegt ein Failure vor. Diese Trennung ist für die Evaluation wesentlich. Das Abziehen des GPS-Empfängers ist die Fehlerinjektion. `have_fix=false` ist ein interner Fehlerzustand. Die Übertragung von `GPS=ERR;MODE=LAST` ist dagegen eine kontrollierte Reaktion, die einen unmarkierten Ausfall der Gesamtfunktion verhindert.

Unterspannung, Brown-out und intermittierende Versorgung. Ein Brown-out ist kein zwingend vollständiger Spannungsverlust, sondern ein Betriebszustand, in dem die

Versorgung unter den für zuverlässige Ausführung erforderlichen Bereich absinkt. Balsamo et al. unterscheiden bei intermittierender Versorgung eine minimale Betriebsspannung sowie getrennte Schwellen für Hibernation und Wiederaufnahme; unterhalb der minimalen Spannung kann ein Prozessor unvorhersehbar arbeiten oder vollständig abschalten [26, S. 15–16]. Ein beobachteter Neustart bei einer am Netzteil eingestellten Spannung ist deshalb nicht automatisch mit der internen BOR-Schwelle des Mikrocontrollers gleichzusetzen.

Forschung zu intermittierend versorgten eingebetteten Systemen zeigt außerdem, dass wiederholte Spannungsausfälle nicht nur Verfügbarkeit, sondern auch Zustandskonsistenz betreffen. Mementos misst die verfügbare Energie und sichert den Ausführungszustand vor einem erwarteten Spannungsverlust in nichtflüchtigem Speicher [27, S. 159–163]. DINO zeigt, dass Neustarts an beliebigen Programmpunkten zu teilweise oder wiederholt ausgeführten Operationen und damit zu inkonsistenten Zuständen führen können [28, S. 575–577]. Die vorliegende Arbeit implementiert kein solches Checkpointing; die Arbeiten dienen als theoretische Grundlage für die Trennung von Spannungserkennung, definiertem Zustand, Neustart und anschließender Recovery.

2.4. Fehlerbehandlung als Folge von Phasen

Eine vollständige Fehlerbehandlung besteht aus mehreren Phasen:

Erkennung: Ein Timeout, eine Ausnahme, ein ungültiger Datenrahmen oder ein Hardwarestatus signalisiert eine Abweichung.

Eingrenzung: Die betroffene Fehlerdomäne wird bestimmt, damit funktionierende Komponenten nicht unnötig zurückgesetzt werden.

Bewertung: Kritikalität, Alter des Zustands und Anzahl aufeinander folgender Fehler werden berücksichtigt.

Recovery: Retry, Reinitialisierung, Dienstneustart, Reset oder Reboot stellen einen definierten Zustand her.

Verifikation: Die ursprüngliche Funktion wird erneut geprüft; ein Prozess gilt nicht bereits durch seine Existenz als funktional. Die Phasenfolge wird in dieser Arbeit als projektspezifische Struktur verwendet. Sie knüpft an Randells Trennung von Fehlererkennung, Zustandswiederherstellung und alternativer Ausführung an [5, S. 220–223].

2.5. Recovery-orientierter Entwurf

Recovery-Oriented Computing betrachtet Hardwarefehler, Softwarefehler und Bedienfehler als unvermeidbare Ereignisse und richtet den Entwurf stärker auf eine kurze Mean Time to Repair aus [2, S. 1 und 5]. Dazu gehören gezielte Recovery-Experimente, Partitionierung von Fehlerdomänen, Defense in Depth und die Beobachtbarkeit von Reparaturmechanismen. Die Idee feingranularer Recovery wird durch Microreboots konkretisiert: Anstatt eine gesamte Anwendung zurückzusetzen, wird möglichst nur die fehlerhafte Komponente neu gestartet [6, S. 31–32]. Dieses Prinzip lässt sich auf die vorliegende Arbeit übertragen. Ein abgestürzter GPS-Dienst wird einzeln neu gestartet; ein LCD-Fehler wird lokal behandelt; ein vollständiger Raspberry-Pi-Neustart bleibt die letzte Eskalationsstufe.

2.6. Grundprinzip eines Watchdogs

Ein Watchdog erwartet ein periodisches Lebenszeichen innerhalb eines vorgegebenen Zeitfensters. Bleibt das Lebenszeichen aus, löst er eine vordefinierte Aktion aus [7,

Abschn. WatchdogSec=, 8, Abschn. WATCHDOG=1, 4, S. 1037–1045, Abschn. 33]. Das Lebenszeichen muss den Fortschritt der überwachten Funktion repräsentieren. Wird ein Hardware-Watchdog bedingungslos in einem Timer-Interrupt aktualisiert, kann eine blockierte Hauptschleife unentdeckt bleiben [11, S. 6-7]. Ein Watchdog erkennt primär Zeitüberschreitungen, nicht automatisch die semantische Korrektheit. Eine fehlerhafte, aber weiterhin schnell laufende Schleife kann den Timer fristgerecht bedienen. Deshalb ergänzt der STM32-Supervisor die reine Zeitüberwachung um Gesundheitszustände mehrerer Subsysteme.

2.7. Watchdog-Ebenen

2.7.1. Anwendungs- und Kommunikationsüberwachung

Auf Anwendungsebene werden Ausnahmen, Zustandsalter, Prüfsummen und Heartbeat-Nachrichten verwendet. Diese Mechanismen können differenzierte Fehlerzustände erzeugen und lokale Recovery auslösen. Sie fallen jedoch aus, wenn die gesamte Ausführungsumgebung blockiert.

2.7.2. Service-Watchdog

Ein Service-Manager kann Prozesse überwachen. Bei systemd aktiviert WatchdogSec= einen Dienst-Watchdog. Der Dienst muss periodisch WATCHDOG=1 über sd_notify() senden; bleibt die Nachricht aus, behandelt der Service-Manager den Dienst als fehlerhaft [7, Abschn. WatchdogSec= und Restart=, 8, Abschn. WATCHDOG=1]. In Verbindung mit Restart= entsteht eine Recovery für Prozesse, die formal existieren, aber keinen Fortschritt mehr machen.

2.7.3. Mikrocontroller-Watchdog

Der STM32L432KC besitzt einen Independent Watchdog, der aus dem Low-Speed-Internal-Oszillator getaktet wird und einen Reset auslöst, wenn der Zähler nicht rechtzeitig aktualisiert wird [4, S. 1037–1045, Abschn. 33, 9, S. 30 und 41]. Die unabhängige Taktquelle reduziert gemeinsame Fehlerursachen mit dem Haupttakt. Ein Window Watchdog kann darüber hinaus auch ein zu frühes Lebenszeichen als Fehler erkennen; er wurde in der Arbeit nicht verwendet.

2.7.4. Systemweiter Hardware-Watchdog

Linux stellt Hardware-Watchdogs über die Watchdog-API und Geräte wie /dev/watchdog0 bereit [3, Abschn. „The simplest API“]. Solange ein Userspace-Prozess den Timer innerhalb des vorgegebenen Zeitfensters aktualisiert, bleibt das System aktiv. Bei einem schweren Hänger läuft der Timer ab und die Hardware löst einen Neustart aus. Diese Ebene schützt gegen Fehler, die ein Service-Watchdog in PID 1 nicht mehr behandeln kann.

2.8. Timeout-Dimensionierung

Die Dimensionierung eines Watchdog-Timeouts ist zunächst ein allgemeines Entwurfsproblem. Der Timeout muss größer sein als der größte im fehlerfreien Betrieb erwartete Abstand zwischen zwei gültigen Fortschrittsmeldungen. Zusätzlich sind zeitliche Schwankungen durch Scheduling, Interrupts und asynchrone Kommunikation sowie eine Sicherheitsreserve zu berücksichtigen [10, S. 4 und 12] [11, S. 16].

Dabei bezeichnet T_WDT den Watchdog-Timeout, $T_max,normal$ den größten erwarteten Abstand zwischen zwei zulässigen Keepalives, T_Jitter die zusätzliche zeitliche

Schwankung und T_{Reserve} eine Sicherheitsmarge. Daraus ergibt sich als vereinfachte Entwurfsregel:

$$T_{\text{WDT}} \geq T_{\text{max, normal}} + T_{\text{Jitter}} + T_{\text{Reserve}} \quad (2.1)$$

Die Beziehung ist keine Normformel und kein formaler WCET-Nachweis, sondern beschreibt die allgemeine Dimensionierungslogik dieser Arbeit. Ein langer Timeout reduziert Fehlalarme, verlängert aber die Reaktion auf echte Hänger; ein kurzer Timeout reagiert schneller, kann bei hoher Last oder blockierendem I/O unnötige Neustarts auslösen. Die konkreten Timeout-Werte des aufgebauten Systems werden erst in Kapitel 4 festgelegt.

2.9. Heartbeat und Supervisor

Ein Heartbeat ist ein Lebenszeichen zwischen Komponenten. Das PING/PONG-Verfahren dieser Arbeit prüft den vollständigen Anwendungsdatenpfad: Empfang, Parsing, Verarbeitung und Antwort. Ein Supervisor aggregiert mehrere Gesundheitsinformationen. Neben einem Zeitstempel kann er Kritikalität, Fehlerbestätigungen, Recovery-Zähler und den globalen Systemmodus speichern. Mehrfachabtastrungen vermeiden, dass ein einzelner transienter Fehler sofort zu einer Zustandsänderung führt. Der STM32 bestätigt LCD-Fehler und -Wiederherstellungen jeweils erst nach mehreren aufeinander folgenden Messungen. Kritische Subsysteme beeinflussen die Freigabe des IWDG-Refresh; ein nicht kritischer Displayfehler führt lediglich zum Zustand DEGRADED.

2.10. Datenintegrität serieller Kommunikation

UART liefert einen Bytestrom ohne inhärente Rahmen- oder Integritätsgarantie. Ein Anwendungsprotokoll benötigt deshalb eine Rahmengrenze, eine maximale Länge und eine Prüfinformation. Die Arbeit verwendet eine ASCII-Zeile mit XOR-Prüfsumme. XOR ist einfach und schnell, erkennt aber deutlich weniger Fehlermuster als gut ausgewählte CRC-Verfahren [12, S. 59–60 und 67–69, 13, S. 145–146].

2.11. I2C und lokale Bus-Recovery

I2C verwendet Open-Drain-Leitungen und adressierte Transfers. Die offizielle Spezifikation beschreibt unter anderem START/STOP-Bedingungen, Acknowledgements und das Bus-Clear-Verfahren [14, Abschn. 3.1.1, 3.1.4–3.1.6, 3.1.16]. Bleibt eine Peripherie oder Leitung in einem fehlerhaften Zustand, darf eine blockierende BUSY-Prüfung die Anwendung nicht dauerhaft anhalten. Die Arbeit begrenzt Wartezeiten, setzt den I2C-Block zurück und erzeugt bei Bedarf Taktpulse zur Busfreigabe, bevor das LCD neu initialisiert wird. Das angeschlossene LCD2004-Zeichenmodul wird über den Adapter als HD44780-basiertes Display angesteuert [19, S. 4]. Der eingesetzte AZDelivery-I2C-Adapter verwendet einen PCF8574-I/O-Expander [19, S. 4]. Die Eigenschaften und der hardwareseitig einstellbare Adressbereich des Expanders sind im NXP-Datenblatt dokumentiert [15, S. 1 und 5–8].

2.12. GPS, NMEA und externe Wetterdaten

Als vollständiges USB-GPS-Gerät wurde ein unter der Vertriebsbezeichnung Binghe VK-162 erworbenes Modul eingesetzt. Das Gerät erschien unter Linux als `/dev/ttyACM0`. Seine interne Empfängereinheit identifizierte sich in den protokollierten USB- und Startmeldungen als Hardware der u-blox-7-Familie beziehungsweise als UBX-G70xx; diese Bezeichnung beschreibt die intern erkannte Einheit und nicht die

Vertriebsbezeichnung des vollständigen USB-Geräts. Die experimentelle Identifikation ist in Anhang A dokumentiert. Die interne Empfängereinheit liefert NMEA-Datensätze über USB. Der RMC-Satz enthält Zeit, Status, Position und Bewegungsinformationen; nur der Status A kennzeichnet gültige Navigationsdaten [16, S. 62]. Die Umrechnung von Grad und Minuten in Dezimalgrad erfolgt nach

$$\varphi_{\text{dez}} = \varphi_{\text{Grad}} + \frac{\varphi_{\text{Minuten}}}{60} \quad (2.2)$$

Der Wetterdienst wird über eine HTTPS-Schnittstelle mit JSON-Antwort angesprochen [17, Abschn. „Weather Forecast API“]. Als externe Abhängigkeit ist er bewusst Teil des Fehlermodells: Netzwerkkonnektivität, DNS/TLS und die Verfügbarkeit beziehungsweise Antwort des Wetterdienstes können unabhängig voneinander ausfallen.

2.13. Messung von Recovery-Zeiten

Für Zeitdifferenzen innerhalb eines laufenden Linux-Systems ist eine monotone Zeitquelle erforderlich, weil die Echtzeituhr durch NTP oder einen manuellen Abgleich springen kann. Die Testskripte verwenden daher `time.monotonic()`. Bei vollständigen Neustarts ist eine externe Messinstanz notwendig, da der zu untersuchende Rechner während des Ausfalls keine kontinuierliche Zeitreihe schreiben kann. Bei kleinen Stichproben werden Mittelwert, Median, Minimum, Maximum und Stichproben-Standardabweichung berichtet.

2.14. Zwischenfazit

Keine einzelne Watchdog-Ebene deckt alle Fehlerklassen ab. Datenfehler, Peripheriefehler, Prozess-Hänger, Firmware-Hänger und Kernel-Hänger benötigen unterschiedliche Erkennungs- und Recovery-Mechanismen. Daraus wird im folgenden Kapitel eine hierarchische Architektur abgeleitet, die lokal beginnt und nur bei schwerwiegenden Fehlern bis zum vollständigen Systemneustart eskaliert.

3. Anforderungen und Systemkonzept

3.1. Anwendungsfall als Testumgebung

Als Testumgebung wird ein eingebettetes System verwendet, das seine Position über einen Binghe VK-162 USB-GPS-Empfänger bestimmt, Wetterdaten für diese Position von einem externen Dienst abrufen und die Informationen auf einem LCD darstellt. Der Raspberry Pi übernimmt netzwerk- und betriebssystemnahe Aufgaben. Der STM32 übernimmt die hardwarenahe Displaysteuerung und eine unabhängige Überwachung der Mikrocontroller- Firmware. Der Datenfluss umfasst GPS-Erfassung, Wetterabfrage, UART-Übertragung und Anzeige. Die Fehlerdomänen und Schnittstellen sind in Abbildung 3.1 dargestellt.

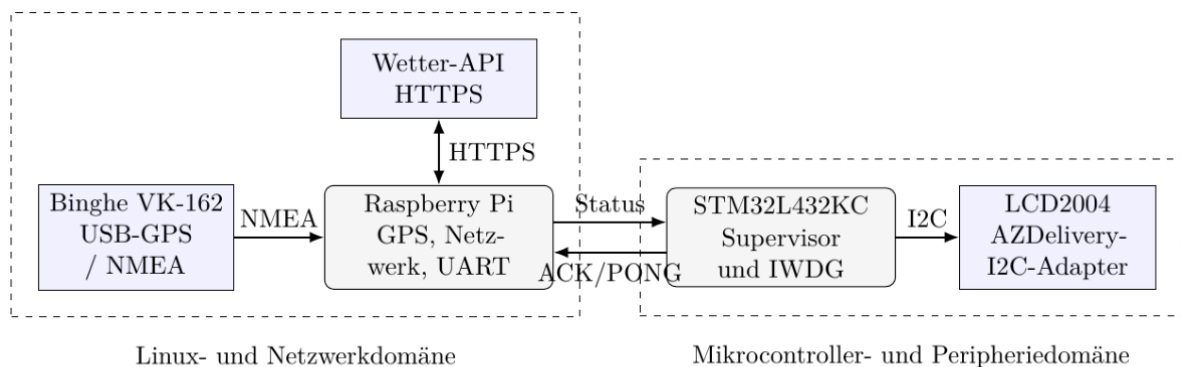


Abbildung 3.1: Gesamtarchitektur und wesentliche Datenpfade des Versuchssystems

Der Anwendungsfall wurde gewählt, weil er mehrere voneinander unabhängige Fehlerquellen in einem überschaubaren Aufbau vereint. Die wissenschaftliche Fragestellung betrifft nicht das Anzeigen von Wetterdaten, sondern die Überwachung und Wiederherstellung des vollständigen Datenpfads.

3.2. Funktionale Anforderungen

Tabelle 3.1: Funktionale Anforderungen

ID	Anforderung
FA-01	Das System muss eine gültige GPS-Position erfassen können.
FA-02	Das System muss Wetterdaten für die zuletzt gültige Position abrufen.
FA-03	Der Raspberry Pi muss GPS-, Netzwerk- und Wetterzustände an den STM32 übertragen.
FA-04	Der STM32 muss gültige Datenrahmen prüfen und auf dem LCD darstellen.
FA-05	Der Raspberry Pi und der STM32 müssen ihre Verbindung über einen Heartbeat prüfen.
FA-06	Fehlerzustände müssen intern protokolliert und gegenüber nachgelagerten Komponenten gekennzeichnet werden.
FA-07	Nach Wegfall einer Störung muss der Normalbetrieb automatisch wiederhergestellt werden.

3.3. Nichtfunktionale Anforderungen

Tabelle 3.2: Nichtfunktionale Anforderungen

ID	Anforderung
NFA-01	Ein Fehler einer Teilfunktion darf andere funktionsfähige Teilfunktionen nicht unnötig stoppen.
NFA-02	Die Recovery muss ohne manuellen Eingriff erfolgen, sofern die Fehlerursache nicht physisch bestehen bleibt.
NFA-03	Nicht kritische Fehler sollen lokal behandelt werden, bevor ein Reset eskaliert wird.
NFA-04	UART-Datenrahmen müssen gegen ungültige Prüfsummen und Pufferüberläufe abgesichert werden.
NFA-05	Hängende Prozesse müssen von abgestürzten Prozessen unterschieden und erkannt werden.
NFA-06	Mess- und Diagnoseinformationen müssen reproduzierbar protokolliert werden.
NFA-07	Die Watchdog-Aktualisierung darf nicht erfolgen, wenn die überwachte Funktion keinen Fortschritt mehr macht.

3.4. Fehlermodell

Das Fehlermodell umfasst Fehler auf Daten-, Komponenten-, Prozess-, Firmware- und Systemebene. Für jede Fehlerklasse werden Erkennung und Recovery bereits im Systementwurf festgelegt.

Tabelle 3.3: Fehlermodell und vorgesehene Recovery-Strategien

Fehler	Ebene	Erkennung	Recovery
Ungültige UART-Prüfsumme	Daten	Prüfsummenvergleich	Rahmen verwerfen, Fehlerantwort senden
UART-Pufferüberlauf	Daten/Link	Längenbegrenzung	Pufferzustand verwerfen und Synchronisation fortsetzen
HTTPS-Verbindung unterbrochen	Netzwerk	Ausnahme bei Wetterabfrage	Fehlerzustand speichern und periodisch erneut versuchen
Kein GPS-Gerät oder kein Fix	Peripherie	Geräte-/Timeout-Prüfung	Letzte gültige Daten verwenden, MODE=LAST
UART-Verbindung unterbrochen	UART-Verbindung	PING/PONG-Timeout	Wiederholungsversuch und automatische Wiederherstellung der Verbindung
LCD getrennt	I2C/LCD	NACK oder Schreibfehler	Lokale LCD-/I2C-Recovery, Modus DEGRADED
Linux-Dienst abgestürzt	Prozess	Prozessende durch systemd	Automatischer Dienstneustart
Linux-Dienst hängt	Prozess	Ausbleibendes systemd-Watchdog-Signal	Prozess beenden und Dienst neu starten
STM32-Firmware hängt	Firmware	IWDG-Timeout	Mikrocontroller-Reset und Bootdiagnose
Linux-Kernel hängt	Linux	BCM2835-Hardware-Watchdog	Vollständiger Neustart des Raspberry Pi
Versorgung fällt aus	System	Neustart nach Rückkehr der Versorgung	Automatischer Boot und Start aller Dienste

Hinweis zur externen Kommunikation: Allgemeine Netzwerkkonnektivität und Verfügbarkeit des Wetterdienstes sind unterschiedliche Fehlerquellen. Die aktuelle Implemen-

tierung fasst beide für den Wetterdatenpfad noch zu einem gemeinsamen Fehlerzustand zusammen; die Konsequenz wird in Abschnitt 4.2.4 erläutert.

3.5. Systemarchitektur

3.5.1. Aufgabenteilung

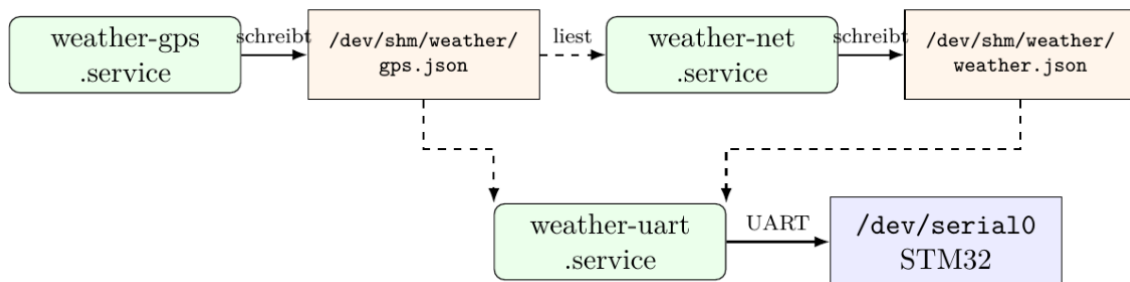
Die Gesamtarchitektur folgt dem Datenpfad GPS -> Wetterabfrage -> UART -> LCD (Abbildung 3.1). Der Raspberry Pi übernimmt GPS-Verarbeitung, externe Kommunikation und Prozessüberwachung; der STM32 übernimmt UART-Validierung, Anzeige und hardwarenahe Überwachung. Die UART-Verbindung bildet die Plattformgrenze. Die folgenden Abschnitte beschreiben die Ebenen in derselben Reihenfolge wie der Datenpfad; Komponentengrenzen werden dabei zugleich als Fehler- und Recovery-Grenzen betrachtet.

Tabelle 3.4: Aufgabenteilung zwischen Raspberry Pi und STM32

Plattform	Aufgaben
Raspberry Pi	GPS-Verarbeitung, Wetterabfrage, Zustandsablage, Dienstverwaltung, systemd-Watchdogs, UART-Master und Hardware-Watchdog des Systems
STM32	UART-Protokollvalidierung, LCD-Ansteuerung, Bootdiagnose, Supervisor, lokale Peripherie-Recovery und IWDG-Reset der Firmware

3.5.2. Dienstarchitektur auf dem Raspberry Pi

Auf dem Raspberry Pi bildet `weather-gps.service` die Eingangsseite des Datenpfads und schreibt den Positionszustand nach `/dev/shm/weather/gps.json`. `weather-net.service` liest diesen Zustand und erzeugt daraus den Wetterzustand in `weather.json`. `weather-uart.service` liest beide Zustände, bildet den seriellen Datenrahmen und besitzt exklusiv `/dev/serial0`. Die Zustandsdateien entkoppeln die Prozesse; ein Fehler eines Dienstes soll deshalb die übrigen Pfade nicht unnötig zurücksetzen. Abbildung 3.2 zeigt diesen gerichteten Datenfluss und die zugehörigen Dienstgrenzen.



Durchgezogene Pfeile: Schreiben bzw. Übertragen; gestrichelte Pfeile: Lesen.

Abbildung 3.2: Entkopplung der Raspberry-Pi-Dienste über atomar aktualisierte Zustandsdateien

3.5.3. Architektur auf dem STM32

Die STM32-Firmware bildet die Ausgangs- und Überwachungsebene des Datenpfads. Sie umfasst Bootdiagnose, UART-Protokollverarbeitung, LCD-Treiber und Supervisor. Der Supervisor verwaltet die Subsysteme `BOOT`, `MAIN_LOOP`, `UART_DRIVER`, `LCD_DRIVER`, `RPI_LINK` und `POWER_SUPPLY`. Für jedes Subsystem werden Zustand, Kritikalität und Alter des letzten erfolgreichen Lebenszeichens betrachtet.

Ein nicht kritischer LCD-Fehler führt zum Modus DEGRADED, während Hauptfunktion und UART-Verbindung weiterlaufen können. Ein Stillstand der Hauptschleife verhindert dagegen die Freigabe des IWDG-Refresh und führt zu einem Reset. Damit bildet der STM32 die lokale Recovery-Ebene unterhalb der Raspberry-Pi-Dienste. Der Datenfluss in Abbildung 3.2 verbindet die drei Dienstgrenzen zu einer Kette: Position, Wetterzustand und serieller Rahmen werden nacheinander erzeugt. Die UART-Schnittstelle bildet anschließend den Übergang zur STM32-Ebene. Dadurch bleibt die Zuordnung von Datenpfad, Fehlerdomäne und Recovery-Ebene über beide Plattformen hinweg nachvollziehbar.

3.6. Konzept der UART-Verbindung

3.6.1. Heartbeat

Der Raspberry Pi sendet periodisch PING. Der STM32 antwortet mit PONG. Dadurch wird nicht nur geprüft, ob die UART-Schnittstelle offen ist, sondern ob die Firmware den Datenrahmen empfangen, verarbeiten und eine Antwort erzeugen kann.

3.6.2. Datenrahmen

Ein Wetterdatenrahmen besitzt grundsätzlich folgende Form:

```
GPS=OK;NET=OK;TEMP=19.3;WIND=10.2;CODE=3;MODE=LIVE*XX
```

Die Nutzdaten werden durch Schlüssel-Wert-Paare beschrieben. Nach dem Sternzeichen folgt eine hexadezimal dargestellte XOR-Prüfsumme. Der STM32 akzeptiert einen Rahmen erst nach erfolgreicher Prüfung. Ein ungültiger Rahmen wird mit ERR:CHECKSUM abgelehnt. Bei Überschreitung des Empfangspuffers wird ERR:BUFFER zurückgegeben.

3.6.3. Betriebsmodi

MODE=LIVE kennzeichnet aktuelle Daten auf Grundlage eines gültigen GPS-Fix. Bei Verlust des GPS-Empfangs bleiben zuletzt bekannte Daten verfügbar und werden mit MODE=LAST sowie GPS=ERR gekennzeichnet. Dadurch bleibt der Systemzustand transparent; veraltete Daten werden nicht als aktuelle Daten dargestellt.

3.7. Hierarchisches Watchdog- und Recovery-Konzept

Das Recovery-Konzept ist als Eskalationshierarchie aufgebaut:

Ebene 1: Datenrahmen verwerfen: ungültige Prüfsumme oder zu langer Rahmen.

Ebene 2: Lokale Komponente reinitialisieren: LCD-/I2C-Fehler.

Ebene 3: Degradierter Betrieb und Retry: GPS- oder Netzwerkfehler.

Ebene 4: Einzelnen Dienst neu starten: Prozessabsturz oder Service-Watchdog-Timeout.

Ebene 5: Mikrocontroller zurücksetzen: Firmware-Hänger des STM32.

Ebene 6: Raspberry Pi neu starten: Kernel-Hänger oder vollständiger Systemfehler.

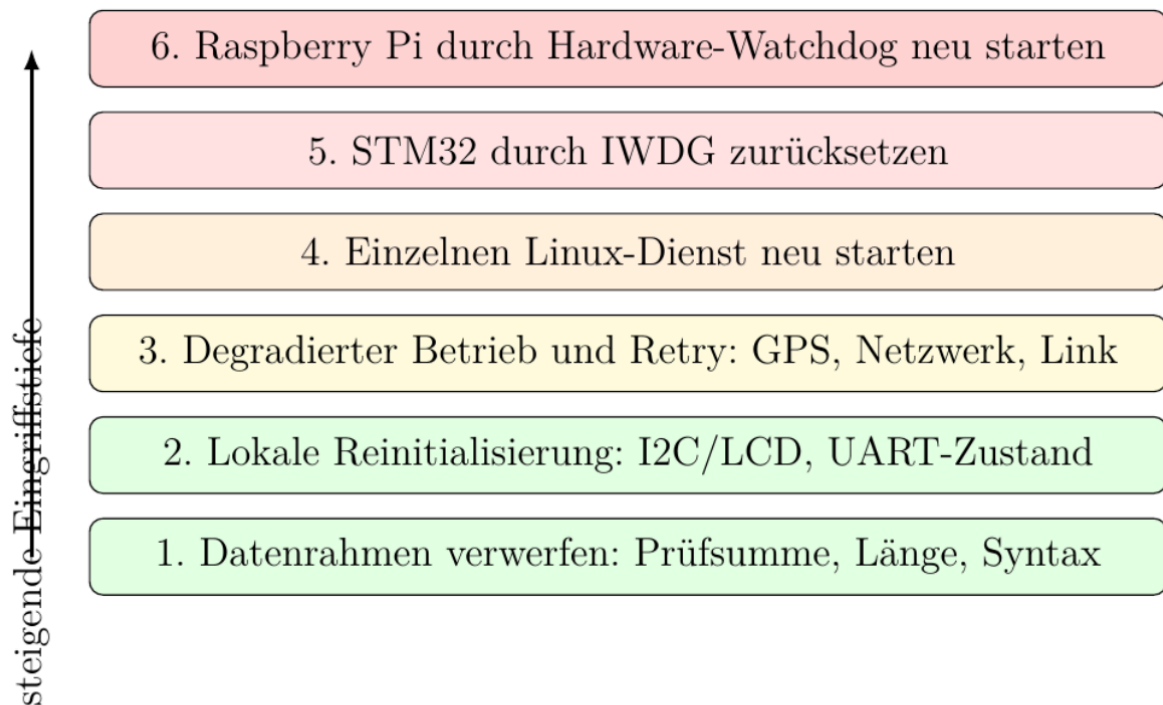


Abbildung 3.3: Eskalationshierarchie der Recovery-Strategien

Diese Reihenfolge minimiert die Eingriffstiefe. Ein nicht kritischer Fehler soll nicht zum Verlust funktionierender Zustände führen. Erst wenn eine lokale Behandlung nicht ausreicht, wird auf eine tiefere Recovery-Ebene eskaliert.

3.8. Systemweiter Recovery-Zustandsautomat

Der globale Systemzustand unterscheidet zwischen stabilen Betriebszuständen und transienten Recovery-Zuständen. Der in Abbildung 3.4 dargestellte Automat ist eine systemweite Abstraktion der verteilten Implementierung. Er ist nicht als zusätzliche monolithische Softwarekomponente zu verstehen: Auf dem STM32 werden BOOT, NORMAL und DE- GRADED durch Bootdiagnose und Supervisor abgebildet; auf dem Raspberry Pi setzen die systemd-Zustände, Restart-Policies und Service-Watchdogs die Übergänge für einzelne Prozesse um. Die Zustände RECOVERY und ESKALATION fassen die jeweils ausgeführten Maßnahmen über beide Plattformen hinweg zusammen.

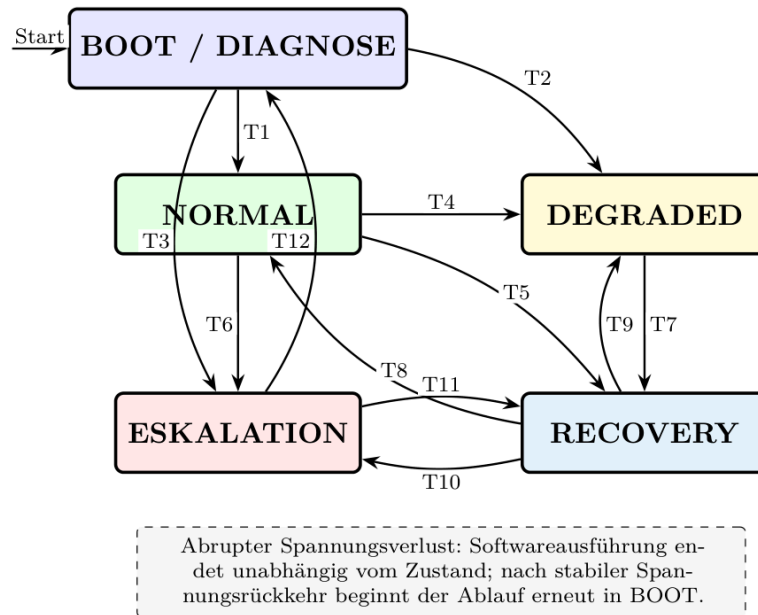


Abbildung 3.4.: Systemweiter Zustandsautomat für Fehlerbehandlung und Recovery

Im Zustand **NORMAL** sind die kritischen Subsysteme gesund und die reguläre Nutzfunktion ist verfügbar. Ein bestätigter Ausfall einer nicht kritischen oder externen Komponente führt zu **DEGRADED**; dort bleiben noch funktionsfähige Pfade aktiv und der Fehler wird nach außen sichtbar gemacht. **RECOVERY** ist ein transienter Zustand. Je nach Fehlerdomäne umfasst er das Verwerfen eines fehlerhaften Rahmens, einen begrenzten Retry, das erneute Öffnen eines Geräts, die I2C-Busfreigabe mit anschließender LCD-Initialisierung oder die Verifikation nach einem Dienstneustart. Der Übergang zurück zu **NORMAL** ist nur nach einem positiven Funktionsnachweis zulässig. Bleibt eine nicht kritische Störung bestehen, führt der Automat zurück nach **DEGRADED**.

ESKALATION wird erreicht, wenn ein kritisches Fortschrittskriterium verletzt wird oder die lokale Recovery innerhalb ihrer festgelegten Grenze erfolglos bleibt. Ein einzelner Linux- Dienst wird zunächst isoliert neu gestartet und anschließend im Zustand **RECOVERY** funktional geprüft. Kann die betroffene Ausführungsbasis selbst keine Recovery mehr durchführen, lösen IWDG beziehungsweise Raspberry-Pi-Hardware-Watchdog einen Reset aus; der nächste ausführbare Zustand ist dann **BOOT**. Damit bildet der Automat das Prinzip der feingranularen Recovery mit kontrollierter Eskalation ab [2, S. 5 und 8; 6, S. 31–32; 11, S. 16].

Tabelle 3.5: Übergänge des systemweiten Recovery-Zustandsautomaten

ID	Übergang	Auslöser beziehungsweise Bedingung
T1	BOOT → NORMAL	Bootdiagnose abgeschlossen; alle für den Normalbetrieb benötigten Prüfungen erfolgreich.
T2	BOOT → DEGRADED	Nicht kritische Komponente oder externe Ressource beim Start nicht verfügbar; Nutzfunktion bleibt möglich.
T3	BOOT → ESKALATION	Kritische Initialisierung scheitert oder ein erforderlicher Fortschrittsnachweis bleibt aus.
T4	NORMAL → DEGRADED	Bestätigter nicht kritischer Fehler, beispielsweise LCD-, GPS-, Netzwerk- oder RPI-Link-Ausfall.
T5	NORMAL → RECOVERY	Fehler ist in der betroffenen Domäne lokal behandelbar.
T6	NORMAL → ESKALATION	Kritisches Lebenszeichen oder der Fortschritt der Hauptfunktion fehlt.
T7	DEGRADED → RECOVERY	Ressource ist wieder verfügbar oder der periodische Retry wird ausgeführt.
T8	RECOVERY → NORMAL	Recovery wurde funktional verifiziert; alle benötigten Pfade sind wieder gesund.
T9	RECOVERY → DEGRADED	Teilfunktion bleibt gestört, der Fehler ist jedoch weiterhin nicht kritisch.
T10	RECOVERY → ESKALATION	Retry-Grenze erreicht, Recovery fehlgeschlagen oder kritischer Fehler besteht fort.
T11	ESKALATION → RECOVERY	Isolierter Dienstneustart ist abgeschlossen; die Nutzfunktion wird anschließend geprüft.
T12	ESKALATION → BOOT	IWDG-Reset des STM32 oder Hardware-Reboot des Raspberry Pi wurde ausgelöst.

3.9. Erfolgskriterien des Gesamtkonzepts

Das Systemkonzept gilt als erfolgreich umgesetzt, wenn die folgenden Punkte experimentell nachgewiesen werden:

- Fehler werden der vorgesehenen Fehlerklasse zugeordnet.
- Die Recovery bleibt auf die kleinste ausreichende Ebene beschränkt.
- Funktionsfähige Dienste bleiben bei Fehlern anderer Dienste aktiv.
- Hängende Software wird auch dann erkannt, wenn der Prozess formal noch existiert.
- Nach Wegfall der Störung wird der funktionale Endzustand ohne manuellen Neustart erreicht.
- Reset- und Neustartursachen sind über Logs oder Diagnoseausgaben nachvollziehbar.

4. Implementierung

Kapitel 3 hat die logische Architektur und die Recovery-Grenzen festgelegt. Dieses Kapitel folgt demselben Datenpfad und beschreibt die konkrete Umsetzung der Schnittstellen, Dienste und Überwachungsmechanismen.

4.1. Hardwareaufbau

Der Hardwareaufbau setzt den in Abbildung 3.1 beschriebenen Datenpfad physisch um: Der GPS-Empfänger ist per USB mit dem Raspberry Pi verbunden, Raspberry Pi und STM32 kommunizieren über UART und der ST steuert das LCD über I2C. Tabelle 4.1 fasst die dafür relevanten Schnittstellen und Parameter zusammen.

Tabelle 4.1: Wesentliche Schnittstellen und Konfigurationswerte

Funktion	Anschluss	Konfiguration
GPS am Raspberry Pi	/dev/ttyACM0	USB-CDC, 9.600 Bd, NMEA
UART am Raspberry Pi	/dev/serial0	115.200 Bd, 8N1
USART1 am STM32	PA9 (TX), PA10 (RX)	Alternate Function 7, 115.200 Bd
I2C1 am STM32	PB6 (SCL), PB7 (SDA)	Alternate Function 4, Open Drain, 100 kHz
LCD2004 mit I2C-Adapter	Adresse 0x27	20 Zeichen × 4 Zeilen, HD44780-kompatibler 4-Bit-Modus
Systemtakt STM32	HSI16	16 MHz

Im Versuchsaufbau wurde für den LCD-Adapter ausschließlich die 7-Bit-I2C-Adresse 0x27 verwendet. Die Registerkonfiguration orientiert sich am STM32-Referenzhandbuch [4, S. 1151-1183, Abschn. 38.1-38.4.10]. I2C-Zugriffe besitzen Software-Timeouts, damit ein dauerhaft gesetztes BUSY-Flag die Hauptschleife nicht unbegrenzt blockiert.

4.2. Raspberry-Pi-Softwarearchitektur

Die Python-Anwendung ist in drei Prozesse aufgeteilt. Jeder Dienst besitzt eine klar abgegrenzte Verantwortung und eine eigene systemd-Überwachung. Diese Partitionierung reduziert den Umfang einer Recovery: Ein Fehler der Wetterabfrage erfordert keinen Neustart des GPS-Empfangs und keinen Verlust der UART-Verbindung.

4.2.1. Gemeinsame Konfiguration

Zentrale Konstanten definieren Schnittstellen, Zeitouts und Zyklen. Der GPS- und Netzwerkdienst laufen mit einer Periode von 10 s, der UART-Dienst mit 5 s. Ein Zustandsdokument wird nach 30 s ohne Aktualisierung als veraltet behandelt. Die letzte Position darf höchstens zwei Stunden alt sein, bevor sie nicht mehr als Fallback verwendet wird. Quelltext 4.1: Auszug der Raspberry-Pi-Konfiguration

```

GPS_PORT = "/dev/ttyACM0"
UART_PORT = "/dev/serial0"
BAUDRATE_GPS = 9600
BAUDRATE_UART = 115200

GPS_POLL_INTERVAL_SECONDS = 10
NET_POLL_INTERVAL_SECONDS = 10
UART_POLL_INTERVAL_SECONDS = 5
STATE_MAX_AGE_SECONDS = 30
MAX_POSITION_AGE_SECONDS = 2 * 60 * 60

USE_UART_CHECKSUM = True
STM32_RESPONSE_TIMEOUT_SECONDS = 2.0
STM32_SEND_RETRIES = 1

```

4.2.2. Zustandsaustausch und atomare Aktualisierung

Die Dienste tauschen kleine JSON-Dokumente in `/dev/shm/weather` aus. Das Verzeichnis liegt auf `tmpfs`, wodurch periodische Schreibzugriffe auf die SD-Karte vermieden werden. Ein Zustand enthält neben Nutzdaten einen Zeitstempel. Die Schreibfunktion erzeugt zuerst eine temporäre Datei und ersetzt anschließend den Zielpfad atomar. Dadurch liest ein anderer Dienst entweder den alten oder den vollständig neuen Zustand, nicht jedoch ein teilweise geschriebenes JSON-Dokument.

Der flüchtige Speicher ist zugleich eine Sicherheitsmaßnahme gegen veraltete Boot-Zustände. Nach einem Neustart existieren keine alten Dateien, die fälschlich als aktuelle Messwerte interpretiert werden könnten.

4.2.3. GPS-Dienst

Der GPS-Dienst öffnet `/dev/ttyACM0`, liest RMC-Sätze und akzeptiert nur Datensätze mit gültigem Status. Breiten- und Längengrad werden aus Grad und Minuten in Dezimalgrad umgerechnet. Ein gültiger Fix aktualisiert `fix_ts`; ein fehlendes Gerät, ein Timeout oder ein ungültiger Status führt zu `have_fix=false`. Der Prozess bleibt aktiv und öffnet das Gerät im nächsten Zyklus erneut. Dadurch ist kein manueller Neustart des Dienstes notwendig, wenn das USB-Gerät zurückkehrt.

4.2.4. Netzwerkdienst

Der Dienst liest eine aktuelle oder noch zulässige letzte Position und ruft die Open-Meteo- Schnittstelle über HTTPS ab [17, Abschn. „Weather Forecast API“]. Für die Fehleranalyse sind drei Ursachen zu unterscheiden: fehlende Netzwerkkonnektivität beziehungsweise DNS/TLS-Fehler, Nichterreichbarkeit oder Fehler des Wetterdienstes sowie eine nicht u auswertbare Antwort. Die aktuelle Implementierung fasst diese Ursachen in `weather.json` noch zu `ok=false` mit `reason=net_error` zusammen und behandelt sie mit demselben periodischen Retry. Diese Vereinfachung wird bei der Interpretation berücksichtigt.

4.2.5. UART-Dienst und Protokollbildung

Nur der UART-Dienst öffnet `/dev/serial0`. Vor jedem Nutzdatenrahmen sendet er PING und erwartet PONG. Anschließend kombiniert er die Zustände der anderen Dienste. Bei Position verfügbar, lautet der Zustand `GPS=ERR;MODE=LAST`. Ein Fehler der Wetterabfrage wird im Protokoll vereinfachend als `NET=ERR` dargestellt; das Feld kennzeichnet damit die Verfügbarkeit der Wetterdaten und nicht ausschließlich die

physische Netzwerkkonnektivität. Die XOR-Prüfsumme wird über die ASCII-Bytes vor dem Stern berechnet:

```
def checksum_xor(text: str) -> int:
    value = 0
    for byte in text.encode("ascii"):
        value ^= byte
    return value

frame = f"{payload}*{checksum_xor(payload):02X}\\n"
```

Der Dienst durchsucht mehrere Antwortzeilen, weil der STM32 vor dem finalen PONG oder OK zusätzliche Diagnosemeldungen wie RPI_LINK_RECOVERED ausgeben kann. Bleibt die Bestätigung aus, wird der Rahmen einmal wiederholt; anschließend wird der Fehler protokolliert, ohne den Prozess zu beenden.

4.3. systemd-Integration

Die zentralen Service-Parameter sind in Quelltext 4.3 zusammengefasst. Alle drei Dienste verwenden denselben Neustart- und Watchdog-Ansatz. Ein gemeinsames weather.target startet und stoppt die drei Prozesse als Gruppe, ohne ihre individuelle Steuerbarkeit aufzuheben. Quelltext 4.3: Wesentliche Einstellungen eines Dienstes

```
[Service]
Type=notify
NotifyAccess=main
User=pi
WorkingDirectory=/home/pi/weather
ExecStart=/usr/bin/python3 -m rpi.service_gps
Restart=always
RestartSec=5
WatchdogSec=30s
```

Die Anwendung liest das von systemd gesetzte WATCHDOG_USEC und sendet das Lebenszeichen typischerweise nach der halben erlaubten Zeit. Die Keepalive-Funktion wird auch während kontrollierter Wartezeiten aufgerufen, damit ein langer I/O-Timeout nicht fälschlich als Prozess-Hänger gilt. Die Semantik von WATCHDOG=1 entspricht der offiziellen systemd-API [7, Abschn. WatchdogSec=, 8, Abschn. WATCHDOG=1].

4.4. Hardware-Watchdog des Raspberry Pi

systemd übernimmt zusätzlich die periodische Aktualisierung von /dev/watchdog0. Die Einstellung RuntimeWatchdogSec= ist in der systemd-Systemkonfiguration dokumentiert [20, Abschn. RuntimeWatchdogSec=]; die Kommunikation mit dem Gerät folgt der Linux- Watchdog-API [3, Abschn. „The simplest API“]. Der Versuch verwendet den BCM2835- Watchdog. In der Konfiguration wurde RuntimeWatchdogSec=10s angefordert;

der aktive Treiber meldete jedoch einen wirksamen Hardware-Timeout von 60 s; der protokollierte wdtctl-Auszug ist in Quelltext C.6 wiedergegeben. Der Kernel- Panic-Test prüft nicht nur die Existenz des Geräts, sondern die vollständige Kette aus ausbleibendem Keepalive, Hardware-Reset, Linux-Boot und automatischem Dienststart.

4.5. STM32-Firmwarestruktur

Die Firmware ist CMSIS-basiert und in Treiber-, Diagnose- und Anwendungsmodulen zerlegt. Zu den Modulen gehören Systemzeit, UART, I2C, LCD2004, Nachrichtenparser,

Supervisor, Power-Monitor, Bootdiagnose und IWDG. CMSIS stellt die standardisierten Cortex-M- Kern- und Registerdefinitionen bereit [21, Abschn. „CMSIS-Core“]; die Peripherieregister werden direkt konfiguriert.

4.5.1. Bootablauf und Diagnose

Nach dem Reset werden Takt, SysTick, UART, I2C, Power-Monitor und Supervisor initialisiert. Die Bootdiagnose prüft I2C/LCD, UART-Kommunikation und den Watchdog. Zusätzlich werden RCC-Resetflags gelesen und anschließend gelöscht. Die Diagnose unterscheidet unter anderem Pin-, Software- und IWDG-Reset. Erst nach Abschluss wechselt der Supervisor vom Boot- in den Laufzeitmodus.

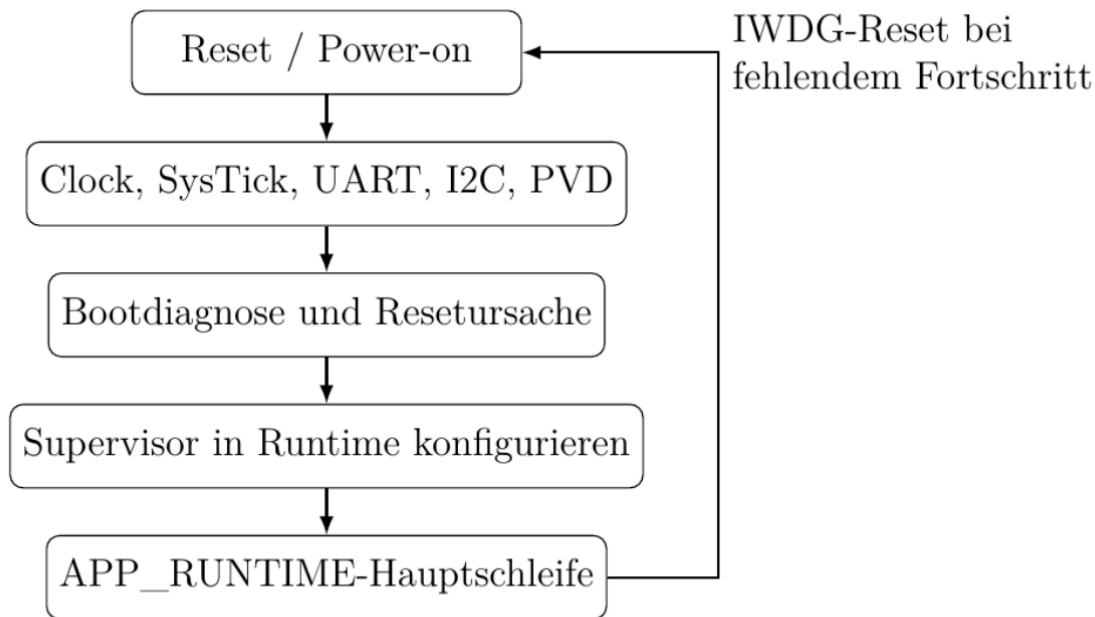


Abbildung 4.1: Vereinfachter Boot- und Recovery-Ablauf des STM32

4.5.2. Supervisor und Gesundheitsmodell

Der Supervisor verwaltet BOOT, MAIN_LOOP, UART_DRIVER, LCD_DRIVER, RPI_LINK und POWER_SUPPLY. Jeder Eintrag enthält Aktivierung, Kritikalität, Gesundheitszustand und Zeitpunkt des letzten Fortschritts. Für kritische Subsysteme gelten Laufzeitgrenzen von typischerweise 3 s; der Raspberry-Pi-Link und das LCD sind nicht kritisch für den IWDG- Refresh. Der globale Modus wird aus den Einzelzuständen abgeleitet. Bei einem bestätigten nicht kritischen Fehler wechselt das System zu DEGRADED. Kritische Fehler verhindern die Freigabe des IWDG-Lebenszeichens. Diagnosebefehle liefern Alter, Zustand und Kritikalität der Subsysteme sowie kumulative Fehler- und Recovery-Zähler. Zusammen mit den Restart- und Watchdog-Zuständen von systemd realisiert diese Logik den in Abbildung 3.4 spezifizierten systemweiten Automaten verteilt über beide Plattformen.

4.5.3. Independent Watchdog

Der IWDG wird mit dem LSI-Oszillator betrieben. Im Normalbetrieb erfolgt ein Refresh nur, wenn `supervisor_all_critical_ok()` wahr ist. Damit ist die Watchdog-Aktualisierung an den Fortschritt der kritischen Funktionspfade und nicht nur an einen periodischen Interrupt gekoppelt. Der Diagnosebefehl `HANG` bestätigt zunächst mit

HANGING, setzt eine verkürzte IWDG- Periode und verbleibt anschließend absichtlich in einer Endlosschleife. Nach dem Reset weist die Bootdiagnose den IWDG als Ursache aus. Der Testpfad ist ausschließlich für die Evaluation vorgesehen.

4.5.4. UART-Empfang und Fehlergrenzen

Die UART-Routine arbeitet nicht blockierend und besitzt eine feste Rahmenlänge. Bei Überschreitung wird der laufende Rahmen verworfen und ERR:BUFFER gesendet. Ein vollständiger Wetterrahmen wird erst nach Prüfung des Stern-Trennzeichens und der XOR- Prüfsumme geparkt. Ein Fehler führt zu ERR:CHECKSUM; ein gültiger Rahmen zu OK. Die Tests U1 und U2 prüfen zusätzlich, ob nach dem Fehler sofort wieder ein PING und ein gültiger Wetterrahmen verarbeitet werden können. Damit wird nicht nur die Erkennung, sondern auch die Wiederherstellung der Parser-Synchronisation bewertet.

4.5.5. LCD- und I2C-Recovery

Die LCD-Gesundheit wird jede Sekunde geprüft. Einzelne Fehlschläge ändern den Zustand noch nicht; erst drei aufeinander folgende Fehler bestätigen den Ausfall. Der Treiber begrenzt I2C-Wartezeiten, setzt die Peripherie zurück und kann bis zu neun SCL-Pulse zur Busfreigabe erzeugen. Bei bestätigtem Ausfall wird LCD_DRIVER=FAIL und der globale Modus DEGRADED gesetzt.

Nach drei erfolgreichen Proben wird das LCD neu initialisiert, der Recovery-Zähler erhöht und der Modus wieder zu NORMAL geändert. Die Konfiguration RESET_AFTER_LCD_STABLE_RECOVERY=0 stellt sicher, dass die Wiederkehr des nicht kritischen Displays keinen vollständigen Mikrocontroller-Reset auslöst.

4.5.6. Spannungsüberwachung

Der STM32-PVD und ein Supervisor-Eintrag POWER_SUPPLY sind in der Firmware vorbereitet. Die Zustandsänderung wird entprellt und kann als kritisch bewertet werden. Ergänzend wurde das Verhalten der vollständigen Nucleo-Platine bei schrittweise abgesenkter VIN-Spannung untersucht.

Reaktive Verfahren wie Hibernus verwenden eine Spannungsschwelle oberhalb des nicht mehr sicheren Betriebsbereichs, damit vor dem eigentlichen Brown-out noch genügend Restenergie für eine definierte Zustandsaktion verbleibt [26, S. 15–17].

4.6. Diagnose, Logs und Reproduzierbarkeit

Die Linux-Dienste schreiben Ereignisse in das systemd-Journal und Status in JSON-Dateien. Der STM32 liefert textbasierte Diagnosen über UART. Die Testskripte sichern vor jeder Fehlerinjektion einen Journal-Cursor und speichern Konsolenausgabe, Rohlogs, CSV-Einzelwerte und statistische Zusammenfassungen in testfallspezifischen Verzeichnissen. Das vollständige Evaluationsverzeichnis wurde anschließend archiviert und mit SHA-256 versehen.

5. Evaluation

Dieses Kapitel untersucht, ob die im Rahmen der Arbeit implementierten Watchdog und Recovery-Strategien unterschiedliche Fehlerzustände zuverlässig erkennen, eingrenzen und automatisch beheben. Betrachtet werden Fehler auf mehreren Ebenen: fehlerhafte Kommunikationsdaten, Ausfälle von Peripherie und externen Verbindungen, abgestürzte oder hängende Linux-Dienste, ein Stillstand der Mikrocontroller-Firmware, ein Kernel- Hänger, ein vollständiger Ausfall der Spannungsversorgung sowie kontrolliert abgesenkte Versorgungsspannungen. Das Ziel der Evaluation besteht nicht ausschließlich im Nachweis einer Fehlererkennung. Ein Test gilt erst dann als erfolgreich, wenn der zuvor definierte funktionale Endzustand wieder erreicht wurde. Dazu zählen je nach Testfall beispielsweise ein gültiger GPS-Fix, eine erfolgreiche Wetterabfrage, ein bestätigter UART-Datenrahmen, der Supervisor- Modus NORMAL oder ein automatisch wiederhergestelltes Gesamtsystem.

5.1. Zielsetzung und Evaluationsfragen

Die Evaluation beantwortet insbesondere folgende Fragen:

1. Werden die vorgesehenen Fehlerzustände zuverlässig erkannt?
2. Wird der Fehlerzustand korrekt an nachgelagerte Komponenten weitergegeben?
3. Wird die jeweils kleinstmögliche geeignete Recovery-Ebene verwendet?
4. Kehrt das System ohne manuellen Eingriff in einen funktionsfähigen Zustand zurück?
5. Wie lange dauern Fehlererkennung, Fehlerweitergabe und funktionale Wiederherstellung?
6. Wie reproduzierbar sind die gemessenen Zeiten über mehrere Versuchsdurchläufe?

5.2. Versuchsaufbau

5.2.1. Hardwareaufbau

Der Versuchsaufbau besteht aus einem Raspberry Pi 4 Model B und einem STM32-Nucleo- L432KC-Entwicklungsboard. Der Raspberry Pi übernimmt die Positionsverarbeitung, die Abfrage der Wetterdaten und die Verwaltung der Linux-Dienste. Der STM32 steuert das Display, validiert die UART-Daten und überwacht mehrere interne sowie externe Systemkomponenten. Die Positionsdaten werden mit einem unter der Vertriebsbezeichnung Binghe VK-162 erworbenen USB-GPS-Empfänger erfasst. Das vollständige Gerät wird unter Linux als serielle Schnittstelle `/dev/ttyACM0` eingebunden und liefert NMEA-Datensätze mit 9.600 Bd. Die intern erkannte Empfängereinheit meldete sich in den Versuchslogs als ublox-7- beziehungsweise UBX-G70xx-Hardware; die zugehörigen USB- und Startmeldungen sind in Anhang A dokumentiert. Aus den GPRMC-Datensätzen werden Breiten- und Längengrad bestimmt. Die Kommunikation zwischen Raspberry Pi und STM32 erfolgt über eine UART- Verbindung mit 115.200 Bd. Auf dem Raspberry Pi wird `/dev/serial0` verwendet. Auf dem STM32 kommt USART1 mit PA9 als TX und PA10 als RX zum Einsatz. Zur Anzeige der Wetter- und Systeminformationen wird ein LCD2004-Modul mit vier Zeilen und jeweils 20 Zeichen verwendet. Der eingesetzte AZDelivery-I2C-Adapter verwendet einen PCF8574-I/O-Expander [19, S. 4]; dessen technische Eigenschaften sind im NXP-Datenblatt doku-

mentiert [15, S. 1 und 5–8]. Das angeschlossene Zeichenmodul wird über den Adapter als HD44780-basiertes Display angesteuert [19, S. 4]. Im Versuchsaufbau wurde ausschließlich die 7-Bit-Adresse 0x27 bei einer Busfrequenz von 100 kHz verwendet. Für den ergänzenden Unterspannungstest wurde ein Labornetzteil verwendet. Die NUCLEO-L432KC-Platine wurde über VIN und GND versorgt; die eingestellte Spannung wurde in 1-V-Schritten von 8 V bis 2 V abgesenkt. Für die Raspberry-Pi-Beobachtung wurde bei einer Sollspannung von 4,50 V der Status mit `vcgencmd get_throttled` abgefragt.

Tabelle 5.1: Hardwarekomponenten des Versuchsaufbaus

Komponente	Funktion
Raspberry Pi 4 Model B	Positionsverarbeitung, Wetterabfrage, Dienstverwaltung, Fehlerbehandlung und Kommunikation mit dem STM32
STM32 Nucleo L432KC	Displaysteuerung, Kommunikationsvalidierung, Bootdiagnose, Supervisor und hardwarebasierte Watchdog-Recovery
Binghe VK-162 USB-GPS-Empfänger	Bestimmung der geografischen Position über NMEA-Datensätze
LCD2004 mit AZDelivery-I2C-Adapter (PCF8574)	Anzeige von Wetterdaten und aktuellem Systemzustand
UART-Verbindung	Bidirektionale Kommunikation zwischen Raspberry Pi und STM32
I2C-Verbindung	Kommunikation zwischen STM32 und LCD-Modul

5.2.2. Softwarearchitektur des Raspberry Pi

Die Anwendungssoftware des Raspberry Pi ist in drei voneinander getrennte systemd-Dienste aufgeteilt:

- `weather-gps.service`,
- `weather-net.service`,
- `weather-uart.service`.

Der GPS-Dienst liest die NMEA-Daten des GPS-Empfängers ein und speichert den aktuellen Positionszustand in `/dev/shm/weather/gps.json`. Der Boolesche Wert `have_fix` zeigt an, ob eine gültige Position verfügbar ist. Der Netzwerkdienst verwendet die zuletzt gültige Position für die Wetterabfrage. Das Ergebnis wird in `/dev/shm/weather/weather.json` gespeichert. `ok=true` kennzeichnet eine erfolgreiche Wetterabfrage. `ok=false` fasst in der aktuellen Implementierung sowohl Fehler der Netzwerkkonnektivität als auch Fehler des externen Wetterdienstes zusammen; diese Ursachen werden bei der Interpretation getrennt betrachtet. Der UART-Dienst besitzt exklusiv die serielle Verbindung zum STM32, führt periodisch PING/PONG aus und überträgt anschließend den aktuellen GPS- und Wetterzustand. Ein typischer Datenrahmen lautet:

```
GPS=OK;NET=OK;TEMP=19.3;WIND=10.2;CODE=3;MODE=LIVE*1B
```

Der Wert hinter dem Sternzeichen ist eine XOR-Prüfsumme. Der STM32 verarbeitet den Datenrahmen nur dann weiter, wenn die Prüfsumme gültig ist. Die drei Dienste sind im gemeinsamen systemd-Target `weather.target` zusammengefasst. Die Trennung erlaubt eine lokale Wiederherstellung einzelner Funktionen. Ein Fehler des GPS-Dienstes erfordert beispielsweise keinen Neustart des Netzwerk- oder UART-Dienstes.

5.2.3. Softwarearchitektur des STM32

Die Firmware des STM32 wurde auf Basis von CMSIS implementiert. Beim Systemstart wird eine Bootdiagnose ausgeführt. Dabei werden insbesondere folgende Punkte geprüft:

- I2C-Kommunikation,
- Erreichbarkeit des LCD-Moduls,
- UART-Kommunikation mit dem Raspberry Pi,
- Funktionsfähigkeit des Independent Watchdog,
- Ursache des vorherigen Resets. Nach der Bootdiagnose wechselt die Firmware in den Applikationsmodus. Ein interner Supervisor überwacht während des Betriebs die logischen Subsysteme MAIN_LOOP, UART_DRIVER, LCD_DRIVER, RPI_LINK und POWER_SUPPLY. Jedem Subsystem sind ein Zustandswert, eine Kritikalität und ein Zeitstempel der letzten erfolgreichen Aktivität zugeordnet. Nicht kritische Fehler, etwa ein Ausfall des Displays, führen zunächst zum Systemmodus DEGRADED. Kritische Fehler oder ein Stillstand der Hauptschleife verhindern dagegen das Aktualisieren des Independent Watchdog, wodurch ein Reset des Mikrocontrollers ausgelöst wird.

5.2.4. Hierarchie der Überwachungs- und Recovery-Ebenen

Das System verwendet eine hierarchische Fehlerbehandlung. Es wird nicht bei jedem Fehler das Gesamtsystem zurückgesetzt. Stattdessen kommt zunächst die kleinstmögliche Recovery-Ebene zum Einsatz, die den Fehler beheben kann.

1. Protokollvalidierung: Ungültige Prüfsummen und zu große UART-Nachrichten werden verworfen.
2. Lokale Komponenten-Recovery: Ein LCD-/I2C-Ausfall wird lokal behandelt; das System arbeitet vorübergehend im Modus DEGRADED.
1. Anwendungsbasierter Fallback: Bei GPS-Verlust werden die zuletzt gültigen Daten mit MODE=LAST weiterverwendet.
2. systemd-Restart: Ein abgestürzter Linux-Dienst wird nach RestartSec automatisch neu gestartet.
3. systemd-Service-Watchdog: Ein noch vorhandener, aber hängender Dienst wird anhand ausbleibender Keepalive-Nachrichten erkannt und neu gestartet.
4. STM32-IWDG: Ein Firmware-Hänger führt zu einem Reset des Mikrocontrollers.
5. Raspberry-Pi-Hardware-Watchdog: Ein Kernel-Hänger führt zu einem Neustart des vollständigen Linux-Systems.
6. Recovery nach Spannungsverlust: Nach Rückkehr der Versorgung starten beide Rechner sowie alle Dienste automatisch neu.

5.3. Messmethodik

5.3.1. Messgrößen

Die gezielte Fehlerinjektion ist ein etabliertes Mittel zur experimentellen Bewertung von Fehlererkennungs- und Recovery-Mechanismen [22, S. 75–76]. Für jeden Testfall wurden mindestens zwei Zeitpunkte betrachtet: der Beginn der Fehlerinjektion und das Erreichen eines definierten funktionalen Endzustands. Abhängig vom Testfall wurden zusätzlich Zwischenzeitpunkte protokolliert: Fehlerinjektionszeitpunkt: Zeitpunkt, zu dem der Fehler ausgelöst oder durch den Bediener bestätigt wurde. Fehlererkennungs-

zeitpunkt: Zeitpunkt, zu dem der zuständige Dienst oder Supervisor den Fehler registrierte. Fehlerweitergabezeitpunkt: Zeitpunkt, zu dem der Fehlerstatus an eine nachgelagerte Komponente übertragen wurde. Recovery-Zeitpunkt: Zeitpunkt, zu dem die zuvor ausgefallene Teilfunktion erneut verfügbar war. Funktionale Wiederherstellung: Zeitpunkt, zu dem der vollständige Datenpfad wieder funktionierte und das Ergebnis bestätigt wurde. Die allgemeine End-to-End-Zeit ergibt sich aus

$$t_{\text{E2E}} = t_{\text{funktional}} - t_{\text{Fehler}} \quad (5.1)$$

5.3.2. Monotone Zeitquelle und Journal-Cursor

Für Zeitdifferenzen innerhalb eines laufenden Linux-Systems wurde überwiegend `time.monotonic()` verwendet [23, Abschn. `time.monotonic()`]. Diese Zeitquelle ist gegen nachträgliche Änderungen der Systemzeit unempfindlich. Vor jeder Fehlerinjektion wurde außerdem ein `systemd-Journal-Cursor` gespeichert. Für die Auswertung wurden ausschließlich Logeinträge nach diesem Cursor berücksichtigt. Dadurch konnten ältere Meldungen nicht fälschlich einem aktuellen Versuch zugeordnet werden. Zu den ausgewerteten Meldungen gehörten unter anderem:

```
INTERNET_ERROR
STM32_CONNECTION_ERROR
STM32: PONG
STM32: OK
Watchdog timeout
Failed with result 'watchdog'
RPI_LINK_RECOVERED
```

5.3.3. Externe Messung bei vollständigen Neustarts

Bei einem vollständigen Neustart kann ein lokal laufendes Messprogramm die Messung nicht fortsetzen. Deshalb wurde für die Tests P1 und P2 ein externer Rechner verwendet. Dieser prüfte die Erreichbarkeit des Raspberry Pi in Intervallen von 0,25 s per ICMP-Ping und protokollierte die Zustandswechsel UP, DOWN und erneut UP. Die externe Nichtverfügbarkeit wurde berechnet als

$$t_{\text{extern}} = t_{\text{UP}} - t_{\text{DOWN}} \quad (5.2)$$

Diese Zeit enthält je nach Testfall die verbleibende Watchdog-Zeit, den Reset, den Bootvorgang und die Wiederherstellung der Netzwerkschnittstelle.

5.3.4. Manuell ausgelöste Hardwarefehler

Die manuell injizierten Hardwarefehler betrafen den Verlust des GPS-Empfängers, die Trennung der UART-Verbindung sowie die Trennung des LCD-/I2C-Pfads. In Abschnitt 5.4 werden diese Testfälle als G1, C1 und L1 definiert. Die Störung wurde jeweils durch manuelles Trennen beziehungsweise Wiederverbinden ausgelöst. Die Zeitmessung beginnt deshalb teilweise mit einer Bedienerbestätigung und nicht mit dem exakten elektrischen Umschaltzeitpunkt.

5.3.5. Wiederholungen und statistische Kenngrößen

Die risikoarmen und reproduzierbaren Tests wurden jeweils fünfmal durchgeführt. Der vollständige Spannungsverlust wurde aufgrund der zusätzlichen Belastung des Dateisystems dreimal getestet. Der Kernel-Hänger wurde zweimal ausgeführt; eine gültige externe Zeitmessung liegt für den zweiten Durchlauf vor. Für die wiederholten Tests

wurden Erfolgsrate, Mittelwert, Median, Minimum, Maximum und Stichproben-Standardabweichung bestimmt. Die Erfolgsrate lautet

$$p_{\text{Erfolg}} = \frac{n_{\text{erfolgreich}}}{n_{\text{gesamt}}} \cdot 100\% \quad (5.3)$$

5.3.6. Dokumentation und Datenintegrität

Für jeden Durchlauf wurden Logdateien, Zustandsdateien und CSV-Messwerte in einem separaten Verzeichnis gespeichert. Nach Abschluss der Evaluation wurde das vollständige Verzeichnis archiviert. Der SHA-256-Prüfwert des finalen Archivs lautet:

```
2e5134c401f3308dd6acbd33e57b0e516af9fa5dfe1a8a47ef6e2a3b633b5bc4
```

5.4. Definition der Testfälle

Tabelle 5.2: Übersicht der untersuchten Testfälle (Teil 1)

ID	Fehlerzustand	Fehlerinjektion	Erwartetes Verhalten	Erfolgskriterium
N2	Ausfall der HTTPS-Wetterabfrage	Blockieren ausgehender TCP-Verbindungen zu Port 443 mit nftables	INTERNET_ERROR, Übertragung von NET=ERR, automatische Rückkehr zu NET=OK	Fehler und Recovery ohne Dienstneustart
G1	Verlust des GPS-Empfängers	Entfernen und erneutes Einstecken des USB-GPS	have_fix=false, Fallback MODE=LAST, anschließend MODE=LIVE	Automatische funktionale Wiederherstellung ohne Dienstneustart
C1	Unterbrechung Raspberry Pi-STM32	Trennen der TX-Leitung des Raspberry Pi zum RX-Eingang des STM32	Verbindungsfehler, danach erneut PONG und Daten-ACK	UART-Datenpfad ohne manuellen Prozessneustart wieder funktionsfähig
H1	Hänger der STM32-Firmware	Senden des Befehls HANG	IWDG-Reset, Bootdiagnose und Rückkehr in APP_RUNTIME	Reset-Ursache IWDG und erneutes PING/PONG bestätigt
S1	Hänger des GPS-Dienstes	Senden von SIGSTOP an den Prozess	systemd-Watchdog-Timeout, Prozessabbruch und Neustart	Neuer PID, Ergebnis watchdog und neuer GPS-Fix
S2	Absturz des GPS-Dienstes	Senden von SIGKILL an den Prozess	Sofortige Fehlererkennung und Neustart nach RestartSec	Neuer PID, Signalfehler und neuer GPS-Fix

Hinweis zu N2: Der Test blockiert gezielt die ausgehende HTTPS-Verbindung zur Wetterabfrage. Er prüft damit die Verfügbarkeit dieses externen Datenpfads; eine separate Diagnose allgemeiner Netzwerkkonnektivität ist nicht Gegenstand dieses Testfalls.

Tabelle 5.2: Übersicht der untersuchten Testfälle (Teil 2)

ID	Fehlerzustand	Fehlerinjektion	Erwartetes Verhalten	Erfolgskriterium
S3	Hänger des UART-Dienstes	Senden von SIGSTOP an den Prozess	systemd-Watchdog-Neustart, PONG, Daten-ACK und RPI_LINK_RECOVERED	Vollständiger UART-Datenpfad wiederhergestellt
U1	Ungültige UART-Prüfsumme	Senden eines Datenrahmens mit absichtlich falscher XOR-Prüfsumme	Antwort ERR:CHECKSUM, danach Annahme eines gültigen Rahmens	Fehlerhafte Nachricht verworfen, UART bleibt funktionsfähig
U2	UART-Pufferlimit	Senden einer Nachricht mit 140 Zeichen	Antwort ERR:BUFFER, danach PONG und Annahme eines gültigen Rahmens	Keine dauerhafte Desynchronisation oder Blockade
L1	Ausfall der LCD-/I2C-Verbindung	Trennen und Wiederverbinden der SDA-Leitung	LCD_DRIVER=FAIL, Modus DEGRADED, danach lokale Recovery	Rückkehr zu LCD_DRIVER=OK und NORMAL ohne Reset
P1	Vollständiger Spannungsverlust	Abruptes Trennen und Wiederherstellen der Versorgung	Automatischer Boot beider Systeme und Start aller Dienste	GPS, Wetter und STM32-Kommunikation ohne manuellen Eingriff verfügbar
P2	Linux-Kernel-Hänger	Kernel-Panic über Magic SysRq bei kernel.panic=0	Neustart durch BCM2835-Hardware-Watchdog	Geänderte Boot-ID und vollständige Systemfunktion ohne Power-Cycle

Ergänzend zum ursprünglichen Katalog wurde der Testfall P3 aufgenommen. P3a charakterisiert das Unterspannungsverhalten der NUCLEO-L432KC-Platine mit angeschlossenem LCD bei schrittweise abgesenkter VIN-Spannung; der qualitative Ablauf wurde fünfmal wiederholt. P3b dokumentiert beim Raspberry Pi den Statuswert von `vcgencmd get_throttled` bei einer am Labornetzteil eingestellten Spannung von 4,50 V.

5.5. Messergebnisse

5.5.1. Gesamtübersicht

Alle wiederholten funktionalen Recovery-Testfälle erreichten eine Erfolgsrate von 100 %. Tabelle 5.3 fasst die wichtigsten End-to-End-Messgrößen dieser Testfälle zusammen. Bei U1, U2 und L1 liegen die Zeiten im Millisekundenbereich. Die manuell ausgelösten Tests G1, C1 und L1 besitzen die in Abschnitt 5.3 beschriebenen Einschränkungen. Der ergänzende Unterspannungstest P3 wird separat ausgewertet, da er ein qualitatives Platinenverhalten und keine End-to-End-Recovery-Zeit beschreibt.

Tabelle 5.3: Zusammenfassung der zentralen Messergebnisse

ID	Szenario	Läufe	Erfolg	Hauptmessgröße	Mittel [s]	Min [s]	Max [s]
N2-F	HTTPS-Verbindung blockiert	5	100 %	Fehlerweitergabe bis NET=ERR	7,36	5,10	8,07
N2-R	HTTPS-Verbindung wiederhergestellt	5	100 %	Funktionale Recovery bis NET=OK	10,10	9,95	10,32
G1-F	USB-GPS getrennt	5	100 %	GPS-Fehlerweitergabe bis MODE=LAST	3,12	0,04	10,85
G1-R	USB-GPS verbunden	5	100 %	Recovery bis MODE=LIVE	6,42	0,04	12,56
C1-F	Pi-STM32-Verbindung getrennt	5	100 %	Bedienerbestätigung bis Fehler	3,80	2,26	5,02
C1-R	UART-Verbindung wiederhergestellt	5	100 %	Funktionale Recovery bis ACK	4,73	0,13	13,30
H1	STM32-Firmware-Hänger	5	100 %	HANG bis APP_RUNTIME	5,70	5,70	5,70
S1	GPS-Dienst hängt	5	100 %	Funktionale Recovery	28,46	27,51	31,62
S2	GPS-Dienst stürzt ab	5	100 %	Funktionale Recovery	5,76	5,62	6,10
S3	UART-Dienst hängt	5	100 %	Funktionale Recovery	33,34	33,07	33,91
U1	Ungültige XOR-Prüfsumme	5	100 %	Ablehnung des ungültigen Rahmens	0,010	0,010	0,011
U2	UART-Pufferüberlauf	5	100 %	Erkennung des Überlaufs	0,018	0,018	0,019
L1-F	LCD-SDA getrennt	5	100 %	Bestätigung bis LCD_DRIVER=FAIL	0,046	0,042	0,048
L1-R	LCD-SDA verbunden	5	100 %	Bestätigung bis LCD_DRIVER=OK	0,044	0,042	0,045
P1	Vollständiger Spannungsverlust	3	100 %	Externe Netzwerknichtverfügbarkeit	32,33	32,00	33,00
P2	Linux-Kernel-Hänger	2	100 %	Einzelmessung der externen Netzwerknichtverfügbarkeit: 35,00 s (Durchlauf 02)	–	–	–

Für P2 wurden zwei funktional erfolgreiche Durchläufe ausgeführt. Eine verwertbare externe Zeitmessung liegt jedoch nur für Durchlauf 02 vor. Der Wert von 35 s ist deshalb ein Einzelwert und kein Mittelwert; Streuungsmaße werden für P2 nicht angegeben.

5.5.2. N2: Ausfall und Wiederherstellung der Wetterverbindung

Zum Auslösen des Netzwerkfehlers blockierte eine nftables-Regel ausgehende HTTPS-Verbindungen. Jeder der fünf Durchläufe führte erwartungsgemäß zum Zustand NET=ERR; weder der Netzwerk- noch der UART-Dienst musste dafür neu gestartet werden. Zwischen Aktivierung der Regel und INTERNET_ERROR lagen im Mittel 3,14 s. Bis der STM32 den Fehlerstatus erhielt, kamen durchschnittlich 4,22 s hinzu. Die End-to-End- Fehlerweitergabe dauerte damit im Mittel 7,36 s. Nach Aufhebung der Sperre lieferte die Wetterabfrage nach durchschnittlich 5,82 s wieder gültige Daten. Weitere 4,28 s später war NET=OK an den STM32 übertragen. Für die vollständige funktionale Wiederherstellung ergibt sich ein Mittelwert von 10,10 s. Die Standardabweichung von lediglich 0,14 s weist darauf hin, dass dieser Ablauf vor allem durch die periodischen Dienstzyklen bestimmt wird.

5.5.3. G1: Verlust und Wiederkehr des GPS-Empfängers

Nach dem Trennen des USB-GPS setzte der GPS-Dienst den booleschen Status have_fix auf false. Der UART-Dienst kennzeichnete die Daten daraufhin mit GPS=ERR und MO-DE=LAST. Vom erkannten Geräteverlust bis zu dieser Weitergabe vergingen durchschnittlich 3,12 s. Nach dem Wiederanschluss stand ein gültiger Fix im Mittel nach 4,22 s zur Verfügung; die Umschaltung auf MODE=LIVE folgte nach weiteren 2,20 s. Insgesamt dauerte die funktionale Wiederherstellung im Mittel 6,42 s. Die Einzelwerte streuen deutlich. Ursachen sind die manuelle Fehlerinjektion, die jeweilige Phase der periodischen Dienstzyklen und der Hot-Start des GPS-Empfängers.

5.5.4. C1: Unterbrechung der Verbindung zum STM32

Für C1 wurde die Sendeleitung des Raspberry Pi zum STM32 manuell getrennt. Alle fünf Unterbrechungen führten zu einer Verbindungsfehlermeldung. Ab der Bedienerbestätigung wurden dafür durchschnittlich 3,80 s benötigt. Nach dem Wiederverbinden traf das erste erfolgreiche PONG im Mittel nach 4,68 s ein. Der anschließend gesendete Wetterrahmen wurde bereits nach weiteren 0,05 s bestätigt. Die Gesamtzeiten lagen zwischen 0,13 s und 13,30 s. Fast die gesamte Streuung entstand vor dem ersten PONG, also durch die zeitliche Lage der Wiederverbindung innerhalb des Retry-Zyklus. Sobald der Heartbeat wieder funktionierte, verlief die Datenübertragung dagegen nahezu konstant.

5.5.5. H1: Firmware-Hänger und STM32-IWDG

Mit HANG wurde die Firmware gezielt angehalten; zuvor bestätigte der STM32 den Befehl durch HANGING. Das Boot-Banner erschien im Mittel 1,23 s später, ein PONG während der Bootdiagnose nach 3,35 s. Vom Absenden des HANG-Befehls bis zum erneuten Applikationsbetrieb vergingen jeweils 5,70 s. Die Diagnose meldete in sämtlichen Durchläufen den IWDG als Reset-Ursache. Nach dem Wechsel in den Applikationsmodus funktionierte auch die PING/PONG-Kommunikation wieder. Dass die gerundeten Werte in allen fünf Messungen übereinstimmen, belegt die Reproduzierbarkeit innerhalb der Auflösung von 0,01 s

5.5.6. S1 und S2: Hängender gegenüber abgestürztem GPS-Dienst

Der direkte Vergleich von S1 und S2 macht den Einfluss der Fehlerart sichtbar. Im Hängerfall existierte der Prozess weiter, lieferte jedoch keine Keepalive-Nachrichten. Systemd konnte erst nach Ablauf des Service-Watchdog-Timeouts reagieren; bis zur wiederhergestellten GPS-Funktion vergingen im Mittel 28,46 s. Nach SIGKILL war das Prozessende dagegen sofort erkennbar. Der Service-Manager musste nur die konfigurierte Neustartverzögerung abwarten und den neuen Prozess starten. Die funktionale Wiederherstellung dauerte deshalb im Mittel lediglich 5,76 s. Gegenüber dem Hänger verkürzte sich die Ausfallzeit um 22,70 s, also ungefähr um den Faktor 4,9.

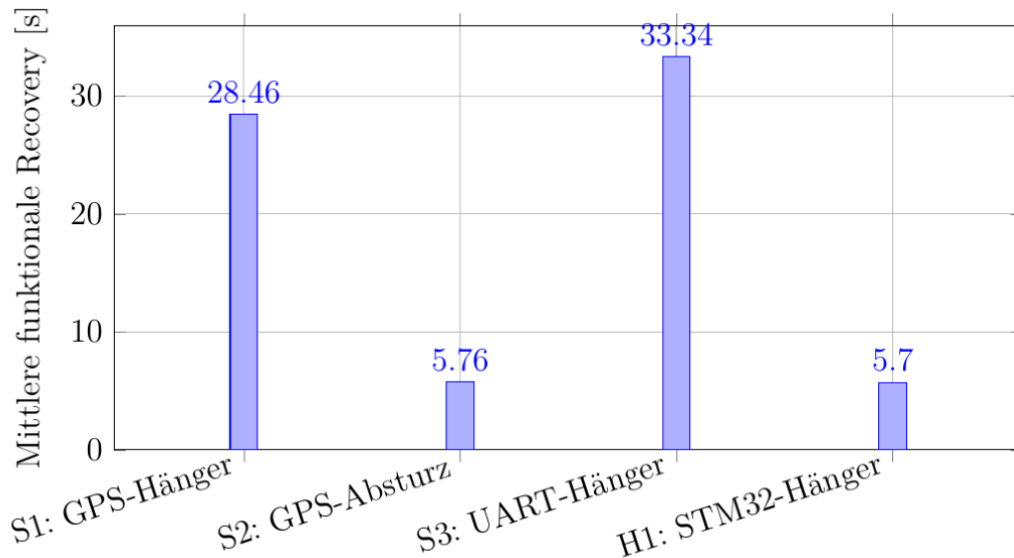


Abbildung 5.1: Vergleich der mittleren funktionalen Wiederherstellungszeiten

5.5.7. S3: Hänger des UART-Dienstes

Auch den angehaltenen UART-Dienst erkannte der systemd-Service-Watchdog in jedem Durchlauf. Bis zum Neustart vergingen durchschnittlich 33,22 s. Danach war die serielle Kommunikation nahezu unmittelbar verfügbar: Das erste PONG traf nach 0,04 s ein, die Bestätigung des Wetterrahmens nach weiteren 0,08 s. Daraus ergibt sich eine mittlere funktionale Wiederherstellungszeit von 33,34 s. Zusätzlich protokollierte der STM32 bei allen fünf Versuchen RPI_LINK_RECOVERED. Damit ist nicht nur der Neustart des Linux-Dienstes, sondern auch die erneute Funktionsfähigkeit des vollständigen Kommunikationspfads nachgewiesen.

5.5.8. U1 und U2: UART-Datenintegrität und Puffergrenzen

U1 bestätigte die Prüfsummenprüfung: Sämtliche fünf Rahmen mit bewusst falschem XOR-Wert wurden verworfen. Bis zur Antwort ERR:CHECKSUM vergingen durchschnittlich 0,010 s. Direkt danach akzeptierte der STM32 jeweils einen gültigen Rahmen und beantwortete auch den folgenden PING. Für U2 wurden Zeilen mit 140 Zeichen gesendet und damit die festgelegte Puffergrenze überschritten. Jeder Versuch erzeugte ERR:BUFFER; die Antwort erfolgte im Mittel nach 0,018 s. Anschließend blieben sowohl PING/PONG als auch die Verarbeitung eines gültigen Datenrahmens funktionsfähig. Die Fehlerbehandlung verwirft somit den betroffenen Rahmen, ohne den Empfänger dauerhaft zu desynchronisieren oder einen Reset auszulösen.

5.5.9. L1: Lokale Recovery des LCD-/I2C-Pfads

Das Trennen der SDA-Leitung führte in jedem Versuch zum Übergang von NORMAL nach DEGRADED; gleichzeitig änderte sich LCD_DRIVER von OK auf FAIL. Nach dem Wiederanschluss kehrte der Supervisor ohne Mikrocontroller-Reset in den Normalzustand zurück. Ab der Bedienerbestätigung lagen zwischen Eingriff und bestätigtem Supervisor- Zustand im Mittel 0,046 s für den Fehler sowie 0,044 s für die Recovery.

5.5.10. P1: Vollständiger Spannungsverlust

Der vollständige Spannungsverlust wurde in drei Durchläufen untersucht. Die externe Netzwerküberwachung erfasste Ausfallzeiten von 33 s, 32 s und 32 s. Der Mittelwert beträgt 32,33 s.

Tabelle 5.4: Externe Netzwerknichtverfügbarkeit bei vollständigem Spannungsverlust

Durchlauf	DOWN	UP	Dauer [s]
P1-01	13.07.2026 03:49:13	13.07.2026 03:49:46	33
P1-02	14.07.2026 00:53:00	14.07.2026 00:53:32	32
P1-03	14.07.2026 00:57:08	14.07.2026 00:57:40	32
		Mittelwert	32,33

Bei jedem Durchlauf änderte sich die Boot-ID; anschließend starteten alle drei Dienste automatisch. GPS-Fix, Wetterabfrage und STM32-Kommunikation wurden ohne manuellen Eingriff wiederhergestellt. Für P1-02 und P1-03 konnten zusätzlich konsistente interne Boot-Messungen ausgewertet werden. Die vollständige Anwendungsbereitschaft lag bei 21,401 s und 21,102 s nach Beginn des Linux-Bootvorgangs. Der Mittelwert der beiden gültigen internen Messungen beträgt 21,252 s. Beim ersten Lauf war die interne Zeitmessung aufgrund einer nachträglichen Korrektur der Systemzeit nicht auswertbar. Die externe Messung blieb davon unbeeinflusst. Nach den abrupten Abschaltungen führte ext4 eine automatische Bereinigung verwaister Dateisystemeinträge durch; schwerwiegende I/O- oder Dateisystemfehler wurden nicht beobachtet.

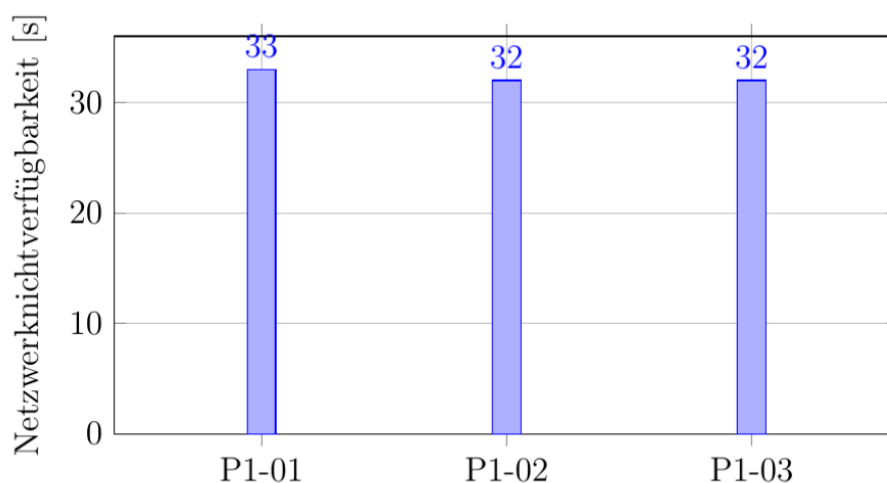


Abbildung 5.2: Extern gemessene Ausfallzeiten der drei P1-Durchläufe

5.5.11. P2: Kernel-Hänger und Hardware-Watchdog

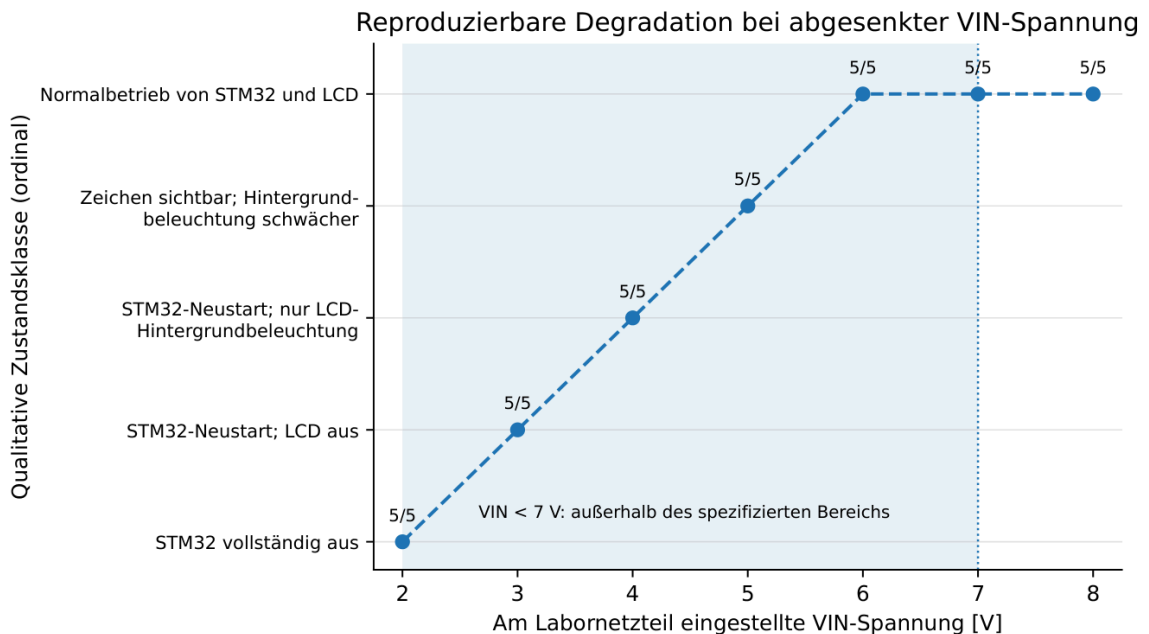
Der Raspberry Pi betrieb den BCM2835-Hardware-Watchdog mit einem von `wdctl` gemeldeten Timeout von 60 s. Der Fehler wurde als Kernel-Panic über Magic SysRq ausgelöst. Da `kernel.panic=0` einen automatischen Neustart des Kernels ausschloss, blieb als vorgesehener Recovery-Pfad nur der unabhängige Hardware-Watchdog.

Beide Durchläufe waren funktional erfolgreich: Die Boot-ID änderte sich, alle drei Dienste starteten erneut, und GPS, Wetterabfrage sowie STM32-Kommunikation standen wieder zur Verfügung. Für die zeitliche Auswertung liegt jedoch nur ein verwertbarer Durchlauf vor: Einzelmessung: 35 s; Der von `wdctl` gemeldete Watchdog-Timeout von 60 s beginnt nicht mit dem ersten fehlgeschlagenen externen Ping. Der Hardware-Watchdog wurde bereits vor der Fehlerinjektion periodisch aktualisiert; beim Auslösen der Kernel-Panic konnte deshalb ein Teil des aktuellen Watchdog- Intervalls bereits verstrichen sein. Die externe Messung setzte zudem erst beim Übergang von Netzwerkzustand UP zu DOWN ein. Der Einzelwert umfasst somit die verbleibende Watchdog-Zeit, den Hardware-Reset, den Linux-Boot und die Wiederherstellung der Netzwerkverbindung.

5.5.12. P3: Kontrollierter Unterspannungstest

NUCLEO-L432KC mit LCD

Die NUCLEO-L432KC-Platine wurde über VIN und GND mit dem Labornetzteil versorgt. Nach Herstellerangabe ist für VIN ein Eingangsbereich von 7 V bis 12 V vorgesehen [24, Abschn. 7.4 und Tab. 5]. Die Werte unterhalb von 7 V lagen damit gezielt außerhalb des spezifizierten Betriebsbereichs. Die Spannung wurde in 1-V-Schritten von 8 V auf 2 V abgesenkt. Der vollständige Ablauf wurde fünfmal wiederholt; in allen fünf Durchläufen wurde dasselbe qualitative Verhalten beobachtet.



Die gestrichelte Verbindung visualisiert nur die Reihenfolge der Messstufen; sie bestimmt keine Schwellenspannung.

Abbildung 5.3.: Qualitative Degradationsstufen in Abhängigkeit von der am Labornetzteil eingestellten VIN-Spannung

Abbildung 5.3 stellt die beobachtete Degradationsfolge auf zwei Achsen dar. Auf der x-Achse steht die am Labornetzteil eingestellte VIN-Spannung. Für die y-Achse wurden

die qualitativen Beobachtungen in fünf ordinal geordnete Zustandsklassen überführt. Die Abstände zwischen diesen Klassen besitzen keine metrische Bedeutung.

Die Beschriftung „5/5“ kennzeichnet, dass der jeweilige Zustand in allen fünf Durchläufen beobachtet wurde.

Tabelle 5.5: Reproduzierbares Unterspannungsverhalten der NUCLEO-L432KC-Platine

VIN	Beobachtung	Durchläufe
8-6 V	STM32 und LCD arbeiteten ohne sichtbare Einschränkungen; Zeichen und Hintergrundbeleuchtung waren normal.	5/5
5 V	Die Zeichen blieben sichtbar; die Hintergrundbeleuchtung wurde erkennbar schwächer.	5/5
4 V	Die Hintergrundbeleuchtung blieb aktiv, Zeichen wurden nicht mehr dargestellt; gleichzeitig wurde ein STM32-Neustart beobachtet.	5/5
3 V	Ein weiterer STM32-Neustart wurde beobachtet; das LCD einschließlich Hintergrundbeleuchtung war vollständig aus.	5/5
2 V	Der STM32 stellte den Betrieb vollständig ein.	5/5

Die Beobachtungen zeigen keinen abrupten Übergang von vollständiger Funktion zu vollständigem Ausfall, sondern eine reproduzierbare Degradationsfolge: zunächst verringerte sich die Helligkeit, anschließend fiel die Zeichendarstellung aus, danach traten Neustarts und schließlich der vollständige Stillstand ein. Ein solcher Bereich zwischen normaler Funktion und Abschaltung entspricht konzeptionell der in der Literatur beschriebenen Unterspannungszone, in der Komponenten bereits unzuverlässig arbeiten können [26, S. 15–16].

Raspberry Pi bei 4,50 V

Beim Raspberry Pi wurde bei einer am Labornetzteil eingestellten Spannung von 4,50 V in Abständen von 0,5 s `vcgencmd get_throttled` aufgerufen. Die erfasste Ausgabe lautete `throttled=0x50005`.

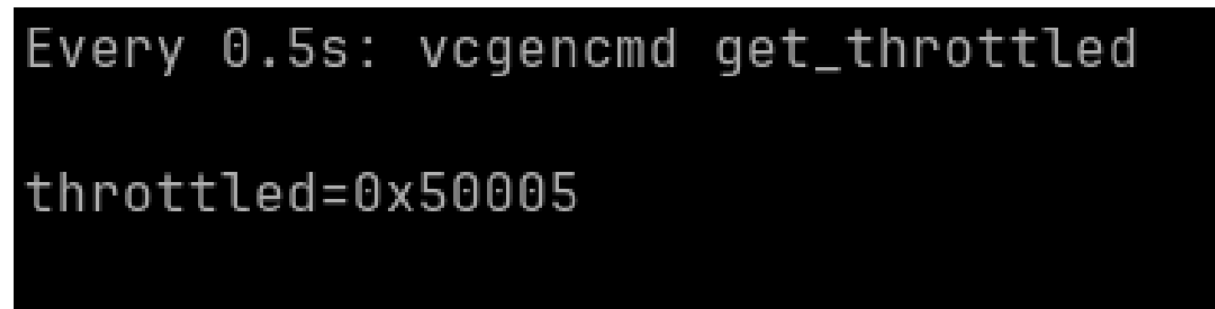


Abbildung 5.4.: Raspberry-Pi-Status bei 4,50 V: `throttled=0x50005`

Nach der offiziellen Raspberry-Pi-Dokumentation stehen die gesetzten Bits 0 und 2 für aktuell erkannte Unterspannung und aktuell aktive Drosselung; die Bits 16 und 18 speichern, dass beide Zustände seit dem letzten Start bereits aufgetreten sind [25, Abschn. “get_throttled”]. Die Messung belegt damit eine zum Aufnahmezeitpunkt aktive Unterspannung und Drosselung.

5.6. Zusammenfassung der Messergebnisse

Innerhalb der beschriebenen Versuchsbedingungen wurden alle fünfmal wiederholten funktionalen Recovery-Testfälle erfolgreich abgeschlossen. Die zentralen Einzel- und Aggregatwerte sind in Tabelle 5.3 zusammengefasst. Für P1 lag die mittlere externe

Netzwerknichtverfügbarkeit bei 32,33 s; für P2 liegt ausschließlich die oben ausgewiesene Einzelmessung von 35 s vor. P3 zeigte in fünf von fünf STM32-Durchläufen dieselbe stufenweise Degradation; die Raspberry-Pi-Messung belegte bei 4,50 V eine aktive Unterspannung und Drosselung. Die Interpretation der Unterschiede, die Beantwortung der Forschungsfragen und die Grenzen der Übertragbarkeit folgen in Kapitel 6.

6. Diskussion

Die folgenden Abschnitte interpretieren die in Kapitel 5 berichteten Messwerte. Aussagen zur Wirksamkeit beziehen sich auf die vorliegende Arbeit, die beschriebenen Fehlerklassen und die durchgeführten Versuchsdurchläufe

Tabelle 6.1: Zuordnung der untersuchten Fehlerklassen zu Recovery-Ebenen

Fehlerklasse	Strategie	Interpretation in der Arbeit
Ungültige Prüfsumme / Pufferüberlauf	Rahmen verwerfen bzw. Empfänger synchronisieren	Fehler bleibt auf Protokollebene isoliert.
LCD-/I2C-Ausfall	Lokale Komponenten-Recovery	Anzeigeausfall stoppt den übrigen Datenpfad nicht.
GPS- oder Netzwerkverlust	Gekennzeichneter Fallback und Retry	Noch verfügbare Funktionen bleiben aktiv.
Prozessabsturz	systemd-Restart	Prozessende ist unmittelbar sichtbar.
Prozess-Hänger	systemd-Service-Watchdog	Liveness muss über ausbleibende Keepalives erkannt werden.
STM32-Firmware-Hänger	IWDG-Reset	Firmware kann keine lokale Recovery mehr ausführen.
Linux-Kernel-Hänger	Hardware-Watchdog	Userspace und systemd sind nicht mehr handlungsfähig.
Vollständiger Spannungsverlust	Automatischer Boot und Dienststart	Recovery beginnt erst nach Rückkehr der Versorgung.

6.1. Beantwortung der Forschungsfragen

6.1.1. F1: Strukturierung und geeignete Fehlerdomäne

Die Versuche sprechen dafür, Fehler auf der niedrigsten Ebene zu behandeln, auf der noch eine funktionsfähige Ausführungsbasis vorhanden ist. Datenfehler werden im Parser verworfen, ein LCD-Fehler im Mikrocontroller lokal behandelt, Prozessfehler durch systemd und ein vollständiger Linux-Hänger durch den Hardware-Watchdog. Die Architektur folgt damit dem Recovery-orientierten Prinzip der Partitionierung und einer mehrstufigen Verteidigung [2, S. 5 und 8]. Die Trennung der Raspberry-Pi-Anwendung in drei Dienste war dafür zentral. Während eines Netzwerkfehlers blieben GPS-Erfassung und UART aktiv. Beim Neustart des GPS-Dienstes musste weder die Wetterlogik noch der serielle Link zurückgesetzt werden. Die Komponentengrenzen wurden somit zugleich zu Recovery- Grenzen.

6.1.2. F2: Eingriffstiefe und Wiederherstellungszeit

Lokale Datenfehler wurden im Millisekundenbereich behandelt. Ein Dienstabsturz benötigte im Mittel 5,76 s, ein Dienst-Hänger 28,46 s. Der STM32-Firmware-Hänger führte nach 5,70 s zurück in den Anwendungsmodus. Vollständige Neustarts verursachten mit etwa 32 s bis 35 s die größte beobachtete Netzwerkunterbrechung. Diese Abstufung zeigt im untersuchten Aufbau, dass die Eingriffstiefe einen praktischen Einfluss auf die Ausfallzeit hat. Feingranulare Recovery erhält funktionsfähige Zustände und entspricht dem Grundgedanken eines Microreboots [6, S. 31–32]. Ein vollständiger Reset bleibt

dennoch unverzichtbar, wenn die betroffene Fehlerdomäne selbst keine Recovery mehr ausführen kann.

6.1.3. F3: Absturz, Hänger, Firmware-Hänger und System-Hänger

Ein Prozessabsturz ist für systemd sofort sichtbar. Die gemessene Wiederherstellung wurde deshalb im Wesentlichen von `RestartSec=5s` bestimmt. Ein mit SIGSTOP erzeugter Hänger ließ den Prozess dagegen formal bestehen. Erst das Ausbleiben von `WATCHDOG=1` führte zum Timeout. Diese Differenz zeigt, weshalb eine Restart-Policy allein nicht gegen Hänger schützt. Beim STM32-Hänger blockierte die vollständige Firmwareausführung; der unabhängig getaktete IWDG blieb als Recovery-Basis erhalten. Beim Kernel-Panic konnte auch systemd nicht mehr reagieren. Erst der BCM2835-Hardware-Watchdog stellte das System wieder her. Die Versuche bilden damit eine Hierarchie zunehmend schwerer Fehlerdomänen ab.

6.1.4. F4: Automatische Rückkehr zum definierten Zustand

Innerhalb des untersuchten Umfangs erreichten alle wiederholten funktionalen Recovery- Tests den vordefinierten Endzustand. Bei GPS und Netzwerk bedeutete dies nicht immer sofort den vollständigen Sollzustand: Das System wechselte zunächst in einen transparenten degradierten Zustand und kehrte nach Wegfall der Störung zu LIVE beziehungsweise NET=OK zurück. Auch Firmware- und Systemneustarts benötigten nach dem Reset keinen manuellen Start der Anwendung. Die Aussage ist auf die getesteten Fehlerklassen und die kleine Stichprobe beschränkt. Fünf erfolgreiche Durchläufe pro risikoarmem Testfall zeigen eine reproduzierbare Funktion innerhalb des untersuchten Aufbaus;

6.2. Dienst-Hänger gegenüber Dienst-Absturz

Der Vergleich S1/S2 ist der deutlichste quantitative Unterschied. Der hängende GPS-Dienst benötigte im Mittel 28,46 s, der abgestürzte Dienst 5,76 s. Der Hänger war damit ungefähr 4,9-mal langsamer wiederhergestellt. Der Unterschied ist kein Mangel der Restart-Policy, sondern eine Konsequenz der fehlenden unmittelbaren Beobachtbarkeit eines Hängers. Die erste S1-Wiederholung dauerte länger als die folgenden Läufe, weil die Fehlerinjektion zu einem anderen Zeitpunkt relativ zum letzten Keepalive erfolgte. Ein Watchdog- Timeout wird ab dem letzten akzeptierten Lebenszeichen berechnet und nicht ab dem extern beobachteten SIGSTOP.

6.3. Nutzen des degradierten Betriebs

Der GPS-Fallback und die LCD-Recovery zeigen, dass ein System trotz eines Teilausfalls noch einen sinnvollen Dienst bereitstellen kann. Bei fehlendem GPS-Empfänger wurden letzte bekannte Daten nur mit ausdrücklicher Kennzeichnung `GPS=ERR;MODE=LAST` weitergegeben. Der LCD-Ausfall beeinträchtigte nicht die UART- und Supervisorfunktion. Dadurch blieb das System diagnostizierbar und konnte die Wiederkehr der Komponente erkennen. Ein degradiertes Modell verlangt jedoch klare Semantik. Veraltete Daten dürfen nicht als aktuell erscheinen, und kritische Fehler dürfen nicht als nicht-kritisch eingestuft werden. In der Arbeit begrenzt `MAX_POSITION_AGE_SECONDS` die Wiederverwendung einer letzten Position.

6.4. Lokale Recovery und Zustandsverlust

Eine Recovery setzt Zustände zurück. Je größer die Recovery-Domäne, desto mehr korrekter Zustand geht verloren. Ein vollständiger STM32-Reset hätte das LCD ebenfalls reinitialisiert, gleichzeitig aber UART-Puffer, Diagnosezustände und laufende Kommunikation unterbrochen. Die lokale I2C-/LCD-Reinitialisierung vermied diesen Kollateraleffekt. Dasselbe Prinzip gilt für Linux-Dienste. Die Zustandsdateien in tmpfs entkoppeln Prozesse, sind aber bewusst flüchtig. Nach einem vollständigen Reboot wird kein alter Zustand übernommen; dafür muss der Datenpfad neu aufgebaut werden. Diese Entscheidung bevorzugt Datenfrische gegenüber Persistenz.

6.5. Datenintegrität und Grenzen der XOR-Prüfsumme

U1 und U2 zeigen innerhalb der durchgeführten Versuche, dass die implementierten Grenzen greifen: Fehlerhafte Prüfsummen und übergroße Rahmen wurden in allen Wiederholungen abgelehnt, und die Kommunikation blieb anschließend synchron. Damit wurde der Schutz gegen die gezielt injizierten einfachen Fehler im Versuchssystem demonstriert. Die XOR-Prüfsumme darf jedoch nicht überinterpretiert werden. Sie erkennt beispielsweise bestimmte Mehrbitfehler nicht und bietet keinen Schutz gegen absichtliche Manipulation. Untersuchungen zu eingebetteten Prüfsummen zeigen die deutlich bessere Fehlererkennungsleistung geeigneter CRCs [12, S. 59–60 und 67–69, 13, S. 145–146].

6.6. Trade-off der Watchdog-Zeiten

Die Timeout-Werte beeinflussen Verfügbarkeit und Fehlalarmrisiko. Ein kürzerer Service-Timeout hätte S1 und S3 beschleunigt, könnte aber bei langsamen Gerätezugriffen oder hoher Systemlast unnötige Neustarts erzeugen. Die Arbeit verwendet 30 s, während die normalen Dienstperioden bei 5 s beziehungsweise 10 s liegen. Für ein Produkt müssten Worst-Case-I/O-Zeiten, CPU-Last, Netzwerk-Timeouts und zulässige maximale Ausfallzeit gemeinsam betrachtet werden. Die Notwendigkeit, den Timeout aus dem zeitlichen Verhalten der gesamten überwachten Software abzuleiten, wird auch in der Watchdog-Literatur hervorgehoben [10, S. 12] [11, S. 16].

Auch beim Hardware-Watchdog dürfen Konfigurationswert und extern beobachtete Ausfallzeit nicht gleichgesetzt werden. wdctl meldete einen Timeout von 60 s; für P2 liegt jedoch nur eine zeitlich auswertbare externe Einzelmessung von 35 s vor. Der Watchdog lief bereits vor der Kernel-Panic und wurde periodisch aktualisiert. Zum Zeitpunkt der Fehlerinjektion konnte daher ein Teil des aktuellen 60 s-Intervalls bereits verstrichen sein. Die externe Messung begann außerdem erst mit dem ersten fehlgeschlagenen Ping und nicht exakt mit der Kernel-Panic. Der Wert von 35 s enthält die verbleibende Watchdog-Zeit, Reset, Bootvorgang und Netzwerkinitialisierung. Er beschreibt ausschließlich die in diesem einen Durchlauf beobachtete End-to-End-Netzwerknichtverfügbarkeit und nicht die reine Watchdog-Auslösezeit.

6.7. Unterspannungsverhalten

Der ergänzende P3-Test zeigt, dass die Energieversorgung eine eigene Fehlerdomäne bildet und dass verschiedene Komponenten nicht gleichzeitig ausfallen müssen. Beim Nucleo-Aufbau war die verringerte LCD-Helligkeit das erste sichtbare Symptom. Danach fiel die Zeichendarstellung aus, während die Hintergrundbeleuchtung noch aktiv blieb; erst bei weiter abgesenkter Eingangsspannung erlosch das LCD vollständig und der STM32 stellte schließlich den Betrieb ein. Dieses Verhalten entspricht einem

degradierten Systemzustand, Der Raspberry-Pi-Wert 0x50005 ergänzt die rein visuelle Beobachtung um einen softwareseitigen Statusnachweis. Die gesetzten aktuellen und gespeicherten Bits zeigen, dass Unterspannung und Drosselung nicht nur früher aufgetreten waren, sondern zum Abfragezeitpunkt aktiv waren [25, Abschn. “get_throttled”]. Die Arbeiten Mementos und DINO verdeutlichen darüber hinaus, dass eine robuste Behandlung wiederkehrender Versorgungsausfälle nicht beim Reset endet: Je nach Persistenzmodell müssen Fortschritt und konsistente Zustandsgrenzen ausdrücklich abgesichert werden [27, S. 159–163; 28, S. 575–579]. Für die vorliegende Arbeit ist daraus vor allem abzuleiten, dass ein zukünftiger Power-Monitor vor Eintritt des unsicheren Bereichs einen definierten Zustand herstellen und dessen Erfolg nach dem Neustart verifizieren sollte.

6.8. Validität und Messunsicherheit

6.8.1. Interne Validität

Automatisierte Tests verwendeten monotone Zeitmessung und Journal-Cursor, um alte Ereignisse auszuschließen. Bei manuellen Leitungsfehlern beginnt die Messung nach Bedienerbestätigung.

6.8.2. Konstruktvalidität

Die funktionale Recovery wurde nicht nur über active/running festgestellt. Ein GPS-Dienst musste einen neuen Fix liefern, der UART-Dienst musste PONG und OK erreichen, und der STM32 musste in APP_RUNTIME zurückkehren. Diese Endpunkte bilden die Nutzfunktion besser ab als reine Prozesszustände.

6.8.3. Externe Validität

Die Hardware, das Betriebssystem und die Netzwerktopologie entsprechen dem in dieser Arbeit. Andere SD-Karten, Kernelversionen, GPS-Empfangsbedingungen oder Wetterdienste können andere Zeiten erzeugen.

6.9. Gesamtbewertung

Das Gesamtkonzept erfüllt die Zielsetzung für den in dieser Arbeit untersuchten Aufbau. Es trennt Fehlerdomänen, macht Zustände beobachtbar und eskaliert die Recovery nach Kritikalität. Keine einzelne Ebene könnte alle Fehler abdecken: systemd kann einen Kernel- Hänger nicht beheben, der IWDG keinen Internetausfall und ein Hardware-Reboot wäre für einen Prüfsummenfehler unverhältnismäßig. Die wichtigste Entwurfserkenntnis lautet daher: Ein robustes heterogenes System benötigt nicht den größtmöglichen Reset, sondern die kleinste hinreichende Recovery-Maßnahme plus eine unabhängige Eskalationsstufe für den Fall, dass diese Maßnahme selbst nicht mehr ausgeführt werden kann.

7. Fazit und Ausblick

7.1. Fazit

Diese Arbeit hat Watchdog- und Recovery-Strategien für ein heterogenes eingebettetes System aus Raspberry Pi und STM32 untersucht. Als Testanwendung wurde ein vollständiger Datenpfad aus GPS-Empfang, Wetterabfrage, UART-Kommunikation und LCD-Anzeige aufgebaut. Der Beitrag liegt nicht in der Wetteranwendung selbst, sondern in der Zuordnung von Fehlerklassen zu abgestuften Recovery-Mechanismen und deren experimenteller Bewertung. Die implementierte Architektur behandelt Datenfehler lokal, reinitialisiert nicht kritische Peripherie, verwendet bei externen Ausfällen einen degradierten Zustand, startet einzelne Linux-Dienste neu und eskaliert erst bei Firmware- oder System-Hängern zu IWDG-Reset beziehungsweise Hardware-Reboot. Die Aufteilung in getrennte systemd-Dienste begrenzt Fehlerdomänen; der STM32-Supervisor verknüpft den IWDG-Refresh mit dem Fortschritt kritischer Subsysteme. Innerhalb der untersuchten Stichprobe waren alle wiederholten funktionalen Recovery-Testfälle in fünf von fünf Durchläufen erfolgreich. Besonders deutlich war der Unterschied zwischen Prozess-Hänger und Prozessabsturz: Der hängende GPS-Dienst benötigte im Mittel 28,46 s, der abgestürzte Dienst 5,76 s bis zur funktionalen Wiederherstellung. Der STM32 kehrte nach einem absichtlichen Firmware-Hänger in 5,70 s zum Anwendungsmodus zurück. Der vollständige Spannungsverlust führte zu einer mittleren externen Netzwerk-Nichtverfügbarkeit von 32,33 s. Der ergänzende Unterspannungstest zeigte bei der Nucleo-Platine in fünf von fünf Durchläufen dieselbe stufenweise Degradation; auf dem Raspberry Pi belegte eine aktive Unterspannung und Drosselung bei der untersuchten Versorgungssituation. Für den in dieser Arbeit untersuchten und die durchgeführten Versuche lassen sich die Forschungsfragen wie folgt beantworten: Fehler müssen auf der kleinsten noch funktionsfähigen Ebene erkannt und behandelt werden; feingranulare Recovery reduziert Zustandsverlust und Ausfallzeit; Hänger benötigen explizite Fortschrittsüberwachung; und die untersuchte unabhängige Watchdog-Kette führte bei den injizierten schweren Fehlern zur automatischen Rückkehr in einen definierten Zustand. Eine einzige Watchdog-Strategie ist für ein solches System nicht ausreichend.

7.2. Ausblick

Für eine Weiterentwicklung bieten sich an:

- quantitative Brown-out- und PVD-Versuche mit definierter Spannungsrampe, Oszilloskopmessung direkt an den Versorgungsschienen, Laststromaufzeichnung und protokollierten RCC-Resetflags,
- Langzeit- und Stresstests mit deutlich größerer Stichprobe,
- Einsatz eines Window Watchdogs oder eines externen Watchdog-ICs,
- CRC, Sequenznummern und explizite Nachrichtenalterung im UART-Protokoll,
- persistente Reset- und Fehlerzähler in einem verschleißbewusst beschriebenen Speicher,
- Read-only-Root-Dateisystem, Overlay-Dateisystem oder Energiepuffer zum Schutz der SD-Karte,

- abgesicherter Bootloader mit signierten Firmware-Updates und Rollback,
- zentrale Telemetrie, automatisierte Alarmierung und Trendanalyse,
- systematische Fault-Injection für Speicher-, Ressourcen- und Timingfehler.

Diese Erweiterungen würden die Übertragbarkeit auf produktive Systeme erhöhen, während das in der Arbeit untersuchte Grundprinzip erhalten bliebe: lokale Recovery zuerst, kontrollierte Eskalation nur bei Bedarf und unabhängige Überwachung für den Ausfall der jeweils darunterliegenden Ebene.

Literaturverzeichnis

- [1] Algirdas Avizienis u. a. „Basic Concepts and Taxonomy of Dependable and Secure Computing“. In: IEEE Transactions on Dependable and Secure Computing 1.1 (2004), S. 11–33. doi: 10.1109/TDSC.2004.2.
- [2] David A. Patterson u. a. Recovery-Oriented Computing (ROC): Motivation, Definition, Techniques, and Case Studies. Techn. Ber. UCB/CSD-02-1175. University of California, Berkeley, 2002 <https://digicoll.lib.berkeley.edu/record/137942> (besucht am 17.07.2026).
- [3] Linux Kernel Documentation Project. The Linux Watchdog Driver API. <https://docs.kernel.org/watchdog/watchdog-api.html> (besucht am 17.07.2026).
- [4] RM0394: STM32L41xxx/42xxx/43xxx/44xxx/45xxx/46xxx Advanced Arm-based 32-bit MCUs—Reference Manual. STMicroelectronics, 2024. https://www.st.com/resource/en/reference_manual/dm00151940-stm32l4x1-advanced-arm-based-32-bit-mcus-stmicroelectronics.pdf (besucht am 17.07.2026).
- [5] Brian Randell. „System Structure for Software Fault Tolerance“. In: IEEE Transactions on Software Engineering SE-1.2 (1975), S. 220–232. doi: 10.1109/TSE.1975.6312842.
- [6] George Candea u. a. „Microreboot—A Technique for Cheap Recovery“. In: Proceedings of the 6th Symposium on Operating Systems Design and Implementation (OSDI '04). USENIX Association, 2004, S. 31–44. https://www.usenix.org/legacy/event/osdi04/tech/full_papers/candea/candea.pdf (besucht am 17.07.2026).
- [7] systemd Project. systemd.service—Service Unit Configuration. <https://www.freedesktop.org/software/systemd/man/latest/systemd.service.html> (besucht am 17.07.2026).
- [8] systemd Project. sd_notify—Notify Service Manager About Start-up Completion and Other Service Status Changes. https://www.freedesktop.org/software/systemd/man/latest/sd_notify.html (besucht am 17.07.2026).
- [9] DS11451: STM32L432KB/STM32L432KC Datasheet. STMicroelectronics, 2018. <https://www.st.com/resource/en/datasheet/stm32l432kc.pdf> (besucht am 17.07.2026).
- [10] Michael J. Pont und Royan H. L. Ong. „Using Watchdog Timers to Improve the Reliability of TTCS Embedded Systems: Seven New Patterns and a Case Study“. In: Proceedings of VikingPLoP 2002. Denmark, 2002. <https://www.cs.le.ac.uk/SoftSD/resources/viking02.pdf> (besucht am 20.07.2026).
- [11] Jacob Beningo. A Review of Watchdog Architectures and their Application to CubeSats. University of Michigan, 2010. <https://www.beningo.com/wp-content/uploads/images/Papers/WatchdogArchitectureReview.pdf> (besucht am 20.07.2026).
- [12] Theresa C. Maxino und Philip J. Koopman. „The Effectiveness of Checksums for Embedded Control Networks“. In: IEEE Transactions on Dependable and Secure Computing 6.1 (2009), S. 59–72. doi: 10.1109/TDSC.2007.70216.
- [13] Philip Koopman und Tridib Chakravarty. „Cyclic Redundancy Code (CRC) Polynomial Selection for Embedded Networks“. In: International Conference on Dependable Systems and Networks. 2004, S. 145–154. doi: 10.1109/DSN.2004.1311885.
- [14] UM10204: I2C-bus Specification and User Manual. NXP Semiconductors, 2021. <https://www.nxp.com/docs/en/user-guide/UM10204.pdf> (besucht am 17.07.2026).
- [15] PCF8574; PCF8574A: Remote 8-bit I/O Expander for I2C-bus with Interrupt. Rev. 5. NXP Semiconductors, 2013. https://www.nxp.com/docs/en/data-sheet/PCF8574_PCF8574A.pdf (besucht am 20.07.2026).
- [16] u-blox 7 Receiver Description Including Protocol Specification V14. GPS.G7-SW-12001-B1. u-blox AG, 2018. https://content.u-blox.com/sites/default/files/products/documents/u-blox7-V14_ReceiverDescriptionProtocolSpec_%28GPS.G7-SW-12001%29_Public.pdf (besucht am 17.07.2026).
- [17] Open-Meteo. Weather Forecast API Documentation. <https://open-meteo.com/en/docs> (besucht am 17.07.2026).

- [18] Raspberry Pi Ltd. Configure UARTs. <https://www.raspberrypi.com/documentation/computers/configuration.html#configuring-uarts> (besucht am 18.07.2026).
- [19] Serielle Schnittstelle I2C für LCD Displays—Datenblatt. AZ-Delivery Vertriebs GmbH. https://cdn.shopify.com/s/files/1/1509/1638/files/Serielle_Schnittstelle_I2C_fur_LCD_Displays_Datenblatt.pdf (besucht am 23.07.2026).
- [20] systemd Project. systemd-system.conf—System and Service Manager Configuration. <https://www.freedesktop.org/software/systemd/man/latest/systemd-system.conf.html> (besucht am 17.07.2026).
- [21] Arm Limited. CMSIS-Core Documentation. https://arm-software.github.io/CMSIS_6/latest/Core/index.html (besucht am 17.07.2026).
- [22] Mei-Chen Hsueh, Timothy K. Tsai und Ravishankar K. Iyer. „Fault Injection Techniques and Tools“. In: Computer 30.4 (1997), S. 75–82. doi: 10.1109/2.585157.
- [23] Python Software Foundation. time.monotonic—Monotonic Clock. <https://docs.python.org/3/library/time.html#time.monotonic> (besucht am 17.07.2026).
- [24] STMicroelectronics. UM1956: STM32 Nucleo-32 Boards (MB1180), User Manual. Rev. 6, 2025. https://www.st.com/resource/en/user_manual/um1956-stm32-nucleo32-boards-mb1180-stmicroelectronics.pdf (besucht am 02.08.2026).
- [25] Raspberry Pi Ltd. Raspberry Pi OS Documentation—vcgencmd, get_throttled. <https://www.raspberrypi.com/documentation/computers/os.html#vcgencmd> (besucht am 02.08.2026).
- [26] Domenico Balsamo, Alex S. Weddell, Geoff V. Merrett, Bashir M. Al-Hashimi, Davide Brunelli und Luca Benini. „Hibernus: Sustaining Computation During Intermittent Supply for Energy-Harvesting Systems“. In: IEEE Embedded Systems Letters 7.1 (2015), S. 15–18. doi: 10.1109/LES.2014.2371494.
- [27] Benjamin Ransford, Jacob Sorber und Kevin Fu. „Mementos: System Support for Long-Running Computation on RFID-Scale Devices“. In: Proceedings of the 16th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS XVI). ACM, 2011, S. 159–170. doi: 10.1145/1950365.1950386.
- [28] Brandon Lucia und Benjamin Ransford. „A Simpler, Safer Programming and Execution Model for Intermittent Systems“. In: Proceedings of the 36th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI ’15). ACM, 2015, S. 575–585. doi: 10.1145/2737924.2737978.